

BACHELOR INFORMATICA



UNIVERSITY OF AMSTERDAM

A Foundation to Formalise Quantum Algorithms in Lean

Jake Bernhard Jongejans

July 1, 2024

Supervisor(s): Māris Ozols

Abstract

Each new quantum algorithm relies on a set of existing subroutines, meaning its correctness relies on the assumption that the previous algorithms are also correct. However, it becomes increasingly infeasible and impossible to adequately test these algorithms, due to computational limits in classical simulation as well as noisy and limited-size quantum hardware, and a lack of access to amplitudes from the underlying quantum state. Even without these constraints, the amount of instances to test grows exponentially as more input qubits are utilised, making it even harder to infer correctness of these algorithms. In the long term, it will become increasingly hard to guarantee the correctness of quantum algorithms and their implementation without either the improvement of quantum hardware or the help of theorem provers. However, there is a lack of a proper foundation to formalise quantum algorithms and prove their correctness in theorem provers, and documentation is, at times, subpar.

Here, I set forth a foundation for formalising quantum algorithms within the Lean 4 programming language and proving their correctness with the Lean theorem prover. This foundation provides core types utilised in defining gates and algorithms within Lean 4, a host of proven mathematical theorems, and proofs of correctness of several standard quantum algorithms to indicate the capabilities of the system. With the foundation built to grow, the intent is to continually improve the foundation and add more algorithms in it, to form a Quantum Lean library, so quantum algorithms with ever-increasing complexity can be formalised and proven correct with the help of computer-aided proofs.

Contents

1	Introduction	7
1.1	Theorem Provers	7
1.2	Proving Correctness of Quantum Algorithms	8
1.3	Source Code	9
2	Quantum Theory	11
2.1	Variable Usage	11
2.2	Qubits	11
2.3	Gates	12
2.3.1	Unitary Matrices	13
2.3.2	Identity Gate	13
2.3.3	Pauli Gates	13
2.3.4	Hadamard Gate	14
2.3.5	Controlled Gates	14
2.3.6	Other Gates	15
2.4	Tensor Powers	16
2.5	Circuits	16
2.6	Algorithms	17
2.6.1	Deutsch	18
2.6.2	Bernstein–Vazirani	19
2.6.3	One-out-of-Four Grover	20
3	Understanding Lean	21
3.1	The Basics of Lean	21
3.1.1	rw	22
3.1.2	rfl and exact	23
3.1.3	simp	24
3.1.4	norm_num	24
3.1.5	Finding Suitable Theorems	25
3.1.6	Proof Efficiency	25
3.1.7	abbrev	26
3.2	Matrices	26
3.2.1	Matrix	26
3.2.2	Operators	27
3.2.3	Kronecker Product	27
3.2.4	reindex	27
3.2.5	Equality and ext	28
3.3	Recursion and Induction	28

4	Creating Quantum Lean	31
4.1	Core Types	31
4.2	Theorems	32
4.2.1	Kronecker Product	32
4.2.2	Tensor Power	34
4.2.3	Reindex	35
4.3	Gates	36
4.3.1	Hadamard	36
4.3.2	Pauli	37
4.3.3	Conditional	37
4.4	Formalising Algorithms	38
4.4.1	Deutsch	38
4.4.2	Bernstein–Vazirani	38
4.4.3	One-out-of-Four Grover	40
4.5	Difficulties	41
4.6	Documentation	41
5	Further Work	43
5.1	Theorems on Tensor Power	43
5.2	Dirac Notation	45
5.3	#eval	45
5.4	Redefining Matrices	46
5.5	Quantum Logic	47
6	Conclusions	49
7	Acknowledgements	51
7.1	AI	51

Introduction

From Deutsch and Bernstein–Vazirani algorithms to quantum Fourier transform, Shor’s algorithm and beyond, the amount of quantum algorithms has steadily increased over time (Jordan 2011). With it, the complexity and size of these algorithms rise, too. As algorithms utilise more qubits, the state space they operate in expands exponentially. Analysing said state space becomes increasingly difficult. Conventional computers can only simulate quantum algorithms up to a certain amount of qubits before the computational limit is reached (Y. Zhou, Stoudenmire, and Waintal 2020). Quantum hardware itself is still error-prone and limited in qubit-count. With these limitations, creating and debugging quantum algorithms becomes ever harder. Finally and clearly, simulation and execution of algorithms only tests for specific cases or a limited amount of times. This limits the certainty of the algorithm’s correctness.

1.1 Theorem Provers

The logical step to take is to avoid trying to prove an algorithm’s correctness through computation in the first place. This is where theorem provers come in. Theorem provers, such as Lean (Lean-FRO 2020), Coq (The Coq Development Team n.d.) and Isabelle (Paulson 1986), aid mathematical proofs. Many mathematical concepts are defined in these theorem provers, along with proven theorems surrounding these concepts. These can be utilised and applied to new proofs of increasingly advanced theorems. With the proofs being computer-aided, and with the help of strict type systems, errors are avoided. Contrary to the computational approach, these proofs have a computationally small footprint, as each proof requires to be proven but once, afterwards becoming a new tool to apply to new proofs. But perhaps most beneficial of all, is that these proofs cover the entire set of inputs that an algorithm can receive, compared to the limit set of inputs usually tested on hardware, of which the results themselves may be flawed due to hardware errors.

At the basis of any theorem prover, we find a typed and functional language, such as OCaml and Haskell. These languages provide the required rigour to handle the complex mathematical ideas, types and structures that are needed to create a consistent system that is able to seamlessly utilise these together. For example, multiplication should work for natural numbers, complex numbers, scalars and matrices, scalars and complex numbers, and so on. Utilising the different attributes of a ring or a semiring needs to be possible in any kind of proof. All of these concepts need to be thought out, and work between concepts, and the language underneath needs to provide a solid foundation to extend these types onto. This becomes the power of a theorem prover.

It should be mentioned that multiple different theorem provers exist, each with a slightly different view on how proofs should be considered and handled, and each with one of more different areas of focus. Each theorem prover will have its own benefits according to the way they designed their software. Some theorem provers will implement features such as *higher-order logic*, *dependent types*, *proof-automation* and *code generation*.

Theorem provers are not limited to the mathematical landscape, either (Hartnett 2016). In cryptography, a neighbouring and related field that likewise handles very mathematical problems, the rise of machine-checked proofs has been very fruitful which can be seen, for example, in the partial automation of quantum cryptographic protocol proofs and the formal proofs of Diffie-Hellman-based protocols (Kubota et al. 2016) (Roy, Datta, and Mitchell 2007). With encryption schemes being integral to the proper workings of society, they need to be safe. Proving their safety is hard, and with the plethora of new algorithms created for post-quantum encryption, many contained bugs, some encryption-breaking and unfixable (NIST 2017).

Dilithium, a candidate in NIST’s Post-Quantum Cryptography Project, was adapted into Draft FIPS 204 to become a new federal information processing standard (Standards and Technology 2023), after thorough testing. However, even Dilithium suffered from a bug, which was fixed with the help of EasyCrypt (Barbosa et al. 2023), a proof assistant aimed at cryptographic proofs (Barthe et al. 2011). Under the hood, these computer-aided provers provide a language in which proofs can be written and automatically verified with the help of computers. As proofs become increasingly hard to get right, the help of computers becomes integral.

In this thesis, Lean was the language of choice. It was initially developed at Microsoft by Leonardo de Moura, and is now open source (Microsoft 2023). The Lean theorem prover (version 4) supports all the aforementioned features, and more (Lean-FRO 2020). With these features, it provides the groundwork to start proving correctness of quantum algorithms. Next to being a theorem prover, it also functions as a programming language (Lean-FRO 2020). The community around Lean is very active and maintains the `mathlib` library that contains a lot of theorems and proofs on a very diverse set of mathematical concepts (Leanprover-Community 2021). For computer scientists, a complete development environment is accessible in the popular Visual Studio Code as an official extension, and tools such as an LSP (Lean-FRO 2021) and theorem search engine Loogle (Breitner 2023) are provided, too, which means implementations within other preferred development environments such as NeoVim are possible with little hassle.

There is a recent momentum for the use of Lean, as it begins to be used for proofs such as:

- The Polynomial Freiman-Ruzsa conjecture over $\mathbb{F}_2$ by Gowers et al.,
- Exponential improvement to the upper bound on Ramsey numbers (Mehta 2023),
- A formalisation of the the Erdos-Graham unit fractions conjecture (Bloom 2021),
- A work-in-progress formalisation of Kelley-Meka bound on Roth numbers (Dillies 2023),
- A future five-year grant to formalise Fermat’s Last Theorem (EPSRC 2023).

1.2 Proving Correctness of Quantum Algorithms

The idea of combining theorem provers and quantum algorithms is a recent idea, though there is already some research to go from. One work formally certified Shor’s factorisation algorithm implementation (Peng et al. 2022). Then there is the paper on quantum Hoare logic (QHL) that allows a way to reason about the correctness of quantum algorithms (J. Liu, Zhan, Wang, Ying, T. Liu, et al. 2019). Finally, there are two implementations of QHL in theorem provers, namely CoqQ, a formal implementation of QHL in the Coq theorem prover (L. Zhou et al. 2023), as well as a formal implementation of QHL into Isabelle (J. Liu, Zhan, Wang, Ying, Tao Liu, et al. 2019). There has also been research into type theory for Hoare logic (TTHL), akin to how type-safe programming languages (e.g. Rust) work compared to type-unsafe languages (e.g. C) (Singhal 2020).

In the scope of this thesis, it is infeasible to implement quantum Hoare logic and by extension type theory for it. However, I will do my best to take QHL into account, so future development can consider its implementation without immediate technical limitations. Next to that are the different functions in QHL, such as `qwhile` and `skip`, which should be possible to create easily in Lean, using its metaprogramming capabilities if necessary. Thus, I believe that the foundation I am creating can be relied upon when future research implements QHL and TTHL in it.

This thesis focuses on laying the groundwork for *Quantum Lean*, a package for Lean that serves as a foundation for formalising quantum algorithms and an algorithm library. This includes the following tasks:

1. Define types in Lean for gates, circuits and qubits;
2. Formalising a set of algorithms that indicate different requirements of the formalisation framework;
 - Deutsch Algorithm: A baseline algorithm to learn Lean’s capabilities and syntax,
 - Bernstein–Vazirani: An algorithm to show Lean can handle arbitrarily-sized algorithms,
 - One-out-of-Four Grover: An algorithm to implement and utilise controlled gates.
3. Documenting Quantum Lean, so further development will require minimal time investment and research into its workings.

As for the layout of this thesis, I first consider basic linear algebra and quantum theory, so all readers are on the same page. I only cover the mathematics and quantum theory that is utilised in the Quantum Lean code, such as tensor powers, qubits, and quantum circuits. This chapter can easily be skimmed by anybody with a bit of prior knowledge in linear algebra and quantum computing.

Next up is an explanation of Lean. I touch upon the basics of Lean for anyone not familiar with the theorem prover and programming language. Furthermore, I give an overview of the `Matrix` type provided by the `mathlib` library. Finally, I speak a bit about induction, which is utilised in variable-sized algorithms.

After that, I write about the Quantum Lean foundation I have been creating. First up, I cover the core types and why they are defined as they are. Then I cover a set of theorems written in Quantum Lean. I continue with the formalisation of algorithms. After that, I indicate the hurdles that had to be overcome in this project. Finally, I write a bit on the documentation I have written on Quantum Lean.

I then write a chapter on future work that can be done on Quantum Lean. Based on the difficulties I have had, there is sections describing possible improvements that can be done both in Quantum Lean, and possibly in `mathlib`. I also point to theorems that future developers can consider to prove, which would be beneficial to Quantum Lean.

Finally, I write my concluding words on this thesis.

1.3 Source Code

The source code of this project can be found on [*GitHub*](#). It is open-source, meaning anybody is allowed to utilise Quantum Lean and contribute to it.

Quantum Theory

This chapter details the mathematical and quantum theories used within this project. I will cover relevant linear algebra, and the basics of quantum computing, as well as the basics of quantum circuit diagrams. In Section 2.6, I will cover the three algorithms formalised in this thesis.

This chapter is meant to provide all the necessary information to understand this thesis, regardless of background. If you are already familiar with some of these theories, I suggest skimming through this chapter to fill any gaps and moving on.

2.1 Variable Usage

To avoid confusion in the usage of variables, this section will define the global types of variables that exist and which groups of letters can represent them. These definitions will be leading, unless specifically stated within the context of a variable's usage.

- $\alpha, \beta, \gamma, \dots \in \mathbb{C}$ represent *complex numbers*;
- $a, b, c, \dots \in \{0, 1\}$ represent *bits*;
- $i, j, k, \dots \in \mathbb{Z}$ represent *integers* (i only when used in tandem);
- $m, n, o, \dots \in \mathbb{N}$ represent *natural numbers*;
- $p, q, r, \dots \in \mathbb{R}$ represent *real numbers*;
- $s, t, u, \dots \in \{0, 1\}^n$ represent *bitstrings*;
- $x, y, z, \dots \in \{1, -1\}$ represent *phases*.

2.2 Qubits

Quantum computing starts with *quantum bits*, or *qubits* for short. Qubits are oft compared to conventional *bits*, which can either be in one of two positions, usually referred to as 0 or 1. A qubit is a little different to this. To explain, we start with a bit of linear algebra. Qubits span a two-dimensional space. This gives us two *basis vectors*, noted as $|0\rangle$ and $|1\rangle$. We define these basis vectors as follows:

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}; \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

A definition of an arbitrary qubit is then as follows. Given complex numbers α and β , a qubit state $|\varphi\rangle$ is a linear combination of the basis vectors, i.e. $|\varphi\rangle = \alpha|0\rangle + \beta|1\rangle$, with the extra constraint that $|\alpha|^2 + |\beta|^2 = 1$.

Unlike *bits*, where we can check their value (0 or 1) without changing its state, we cannot do this with qubits. For qubits, we need to *measure* them. Given α and β , the probability that

$|\varphi\rangle$ measures as $|0\rangle$ is $|\alpha|^2$, and the probability that it measures to $|1\rangle$ is $|\beta|^2$. As is obvious to see, the only two measurement outcomes we are able to read are an equivalent of 0 and 1, but contrary to conventional bits, the state of the qubit is collapsed to the measured value, i.e. before measurement, $|\varphi\rangle = \alpha|0\rangle + \beta|1\rangle$, but after measurement, it is

$$\begin{cases} |0\rangle & \text{with probability } |\alpha|^2, \\ |1\rangle & \text{with probability } |\beta|^2. \end{cases}$$

When we introduce multiple qubits, we can first define the product states of our basis vectors $|0\rangle$ and $|1\rangle$. These are $|0\rangle \otimes |0\rangle$, $|0\rangle \otimes |1\rangle$, $|1\rangle \otimes |0\rangle$ and $|1\rangle \otimes |1\rangle$. A shortened way of writing these is $|a\rangle \otimes |b\rangle = |ab\rangle$. To calculate the corresponding vectors, we utilise the *Kronecker product*, denoted with the $\otimes$ symbol (although Lean uses the notation $\otimes_k$). The Kronecker product works as follows:

$$|0\rangle \otimes |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} \\ 0 \cdot \begin{bmatrix} 1 \\ 0 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

When applied to all pairs, we end up with these:

$$|00\rangle = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}; \quad |01\rangle = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}; \quad |10\rangle = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}; \quad |11\rangle = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

This can be done with an arbitrary number of qubits, of course. The binary string inside the *ket* (the $|$ and $\rangle$) can then be seen as the index of the 1 in the vector. So $|101010\rangle$ has its 1 entry in the 42th place. Note that the state of six qubits already needs to be represented with $2^6 = 64$ entries in its vector. When generalised, a vector for n qubits has the size of 2^n .

These qubit states only constitute the *basis vectors*, and it is possible to create an arbitrary qubit state using a linear combination of these basis vectors, akin to the definition of $|\varphi\rangle$ in the one-qubit example, with α_i as amplitudes:

$$|\varphi\rangle = \sum_{i=0}^{2^n} \alpha_i |i\rangle; \quad \sum_{i=0}^{2^n} \alpha_i^2 = 1.$$

A common two-qubit state used in quantum computing is the $|\Phi^+\rangle$ *Bell state*, which equals $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. This is a *maximally-entangled state*: when the first qubit is measured as 0, the second is equal to 0 as well, and if the first is measured as 1, the second will also be equal to 1 once it is measured.

Its vector notation would be the following:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = \frac{1}{\sqrt{2}} \left(\begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \right) = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}.$$

2.3 Gates

The mathematical basis of quantum computing is linear algebra, and the foundation of quantum circuits is the gates that form them. Gates allow us to do different elementary operations on qubits, to alter their state in a simple way. It is possible to decompose any complicated operation into the set of elementary operations. Just as conventional NOR and NAND gates are functionally complete or *universal* gates, there are sets of quantum gates that form a functionally complete set.

As usual, I will loosely compare a set of these gates to their *conventional* counterparts, as to get a better sense of their effect on qubits. I will include a short quantum circuit in each

subsection to briefly show how each of the gates will look like in a circuit. An empty circuit looks as one or multiple horizontal lines that indicate a qubit, as seen in Figure 2.1.

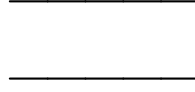


Figure 2.1: An empty circuit containing two qubits.

Before all that, I will first explain some basic properties that each matrix should have for it to be applicable in a quantum circuit, in Subsection 2.3.1.

2.3.1 Unitary Matrices

All gates in quantum computing need to be *unitary*. Unitary matrices M are a subset of matrices that are square, has an inverse M^{-1} , and its inverse is equal to its conjugate transpose $M^{-1} = M^\dagger$. Based on the last requirement, $M^\dagger M = M M^\dagger = I$ should hold.

A conjugate transpose of a matrix M can be determined by transposing the matrix ($M^\top$) and applying the complex conjugate entry-wise. The complex conjugate of a complex number $p + qi$ is $p - qi$. That means that the conjugate transpose of arbitrary square matrix M should look as follows:

$$M^\dagger = \begin{bmatrix} p_{00} + q_{00}i & p_{01} + q_{01}i \\ p_{10} + q_{10}i & p_{11} + q_{11}i \end{bmatrix}^\dagger = \begin{bmatrix} p_{00} - q_{00}i & p_{10} - q_{10}i \\ p_{01} - q_{01}i & p_{11} - q_{11}i \end{bmatrix}.$$

The requirement that the inverse of the matrix should equal its conjugate transpose makes it easy to compute the inverse of a matrix.

2.3.2 Identity Gate

It may be plain and obvious to mention the identity gate, usually indicated with $\mathbb{I}$ (or 1 in Lean). Its use is simple: do nothing. An identity gate can be applied anywhere without modifying the quantum state. Its matrix form is the identity matrix:

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

While the utilisation of an identity matrix may be redundant, it becomes useful when describing the result of a circuit. When writing proofs, we need to be able to state that $AA^{-1} = I$ and that $I \cdot A = A$ for any given matrix A . In this sense, the identity gate becomes integral in our proofs.

In a quantum circuit, the identity gate is not displayed, as it does not do anything. Therefore, it is identical to the line of the circuit, as seen in Figure 2.2.



Figure 2.2: An identity gate on a single qubit.

2.3.3 Pauli Gates

The Pauli gates are an equally integral set of gates. In quantum computing, they are usually denoted as the X, Y and Z gates. Their matrix forms are as follows:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}; \quad Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}; \quad Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

The X gate also bears a different name, the NOT gate. It flips the state $|0\rangle$ to be $|1\rangle$ and vice versa. Since these are the basis vectors from which each qubit state is defined as a linear combination,

the NOT gate extends by linearity to any qubit state: $X(\alpha|0\rangle + \beta|1\rangle) = \alpha|1\rangle + \beta|0\rangle$. Its three circuit forms can be found in Figure 2.3.



Figure 2.3: Three forms of visualisation of the X gate on a circuit.

The Y gate does something similar to the X gate in that it maps $|0\rangle$ to $i|1\rangle$ and $|1\rangle$ to $-i|0\rangle$. Its circuit form is found in Figure 2.4.

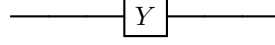


Figure 2.4: The Y gate on a circuit.

The Z gate is sometimes referred to as a flip in phase. It leaves $|0\rangle$ unchanged, while it turns $|1\rangle$ negative, into $-|1\rangle$. Its circuit forms can be found in Figure 2.5.

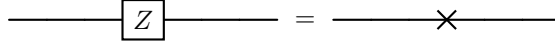


Figure 2.5: Two ways to visualise a Z gate on a circuit.

All three of these gates can be squared to form the identity matrix:

$$X^2 = Y^2 = Z^2 = I.$$

2.3.4 Hadamard Gate

As mentioned in the previous section, we also have a Hadamard gate

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}.$$

The Hadamard gate maps the basis vectors $|0\rangle$ and $|1\rangle$ into two superpositions, usually indicated as $|+\rangle$ and $|-\rangle$ respectively:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{2}; \quad |-\rangle = \frac{|0\rangle - |1\rangle}{2}.$$

When measuring these qubits $|+\rangle$ and $|-\rangle$, there is an equal chance of measuring 0 or 1, as they are *equal* superpositions. Because of this, the Hadamard gate is widely used in quantum computing. A Hadamard gate is indicated on a circuit in Figure 2.6.

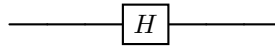


Figure 2.6: A Hadamard gate on a circuit.

2.3.5 Controlled Gates

Up to now, I have only shown gates that function on a single qubit. However, it is also possible to create gates that function on multiple qubits at the same time. Controlled gates are one form of these.

In general, a *controlled gate* can be defined for any arbitrary single-qubit gate M and its values $m_{ij} \in \mathbb{C}$, with its form becoming the following

$$M = \begin{bmatrix} m_{00} & m_{01} \\ m_{10} & m_{11} \end{bmatrix}; \quad CM = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & m_{00} & m_{01} \\ 0 & 0 & m_{10} & m_{11} \end{bmatrix}.$$

The most commonly used controlled gate used in quantum algorithms would be the *controlled X gate*. It is usually referred to as CX or CNOT. Its matrix form would become this:

$$CX = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

On the set of four product basis vectors, it acts like so:

$$|00\rangle \mapsto |00\rangle; \quad |01\rangle \mapsto |01\rangle; \quad |10\rangle \mapsto |11\rangle; \quad |11\rangle \mapsto |10\rangle.$$

As can be seen, when the first qubit is 0, the second qubit remains identical, while if it is 1, the second qubit flips. Hence, we control whether the gate functions based on the value of the first qubit. This is the behaviour of a controlled gate for any chosen gate M .

As for the visualisation of a controlled gate in a circuit, there is a general notation and several special cases for the most common gates. We start with the generic controlled gate in Figure 2.7, which shows the controlling qubit (the first one) with a filled black dot, which is connected to the generic M gate which it is controlling.

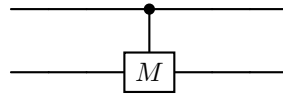


Figure 2.7: A generic controlled gate on a circuit.

Next up are some special cases. The CX gate in Figure 2.8 can utilise the $\oplus$ symbol to become more compact.

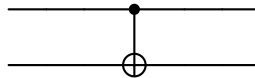


Figure 2.8: A compact representation of the controlled X gate.

We also have a special case for the CZ gate, the controlled Z gate, in Figure 2.9.

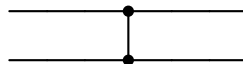


Figure 2.9: A compact representation of the controlled Z gate.

2.3.6 Other Gates

There are other gates in quantum computing, such as the SWAP gate that swaps the values of two qubits, and a double-controlled X or CCX gate, that only negates the target qubit when *both* control qubits are set to 1. It is possible to define any arbitrary gate with a *universal* set of gates. An example of such a set of gates would be the H and CCX gates. Another set is H, CX, S and T, where S and T are described as follows:

$$S = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} \quad T = \begin{bmatrix} 1 & 0 \\ 0 & e^{\frac{i\pi}{4}} \end{bmatrix}.$$

The SWAP gate's decomposition is seen in Figure 2.10, and the CCX gate's special circuit notation can be seen in Figure 2.11.

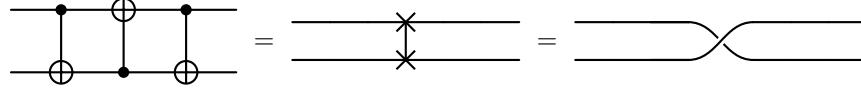


Figure 2.10: A decomposition of the SWAP gate.

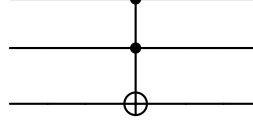


Figure 2.11: A CCX gate.

2.4 Tensor Powers

A gate applies to a limited number of qubits. A Hadamard gate acts on a single qubit, and a CNOT gate acts on two. However, it is possible to apply single qubit gates individually on many qubits simultaneously. By utilising a *tensor power*, it is possible to apply a single gate, such as H or X , to n qubits all at once.

To apply individual gates on multiple qubits at the same time, we can utilise the Kronecker product again. For example, $H \otimes H$ acts on two qubits, applying a Hadamard gate upon both the first and second qubit. This can be seen in Figure 2.12.

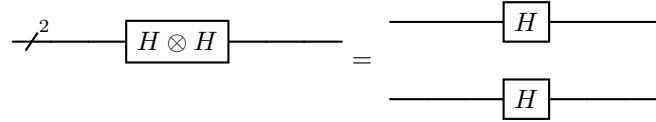


Figure 2.12: A circuit showing $H \otimes H$ being applied is equivalent to applying H to the first and the second qubit.

If we want to simultaneously apply H to an arbitrary number of qubits, which we will do in the Bernstein–Vazirani algorithm later on, we utilise the *tensor power*, $H^{\otimes n}$. Its definition is simple:

$$H^{\otimes n} = \overbrace{H \otimes H \otimes \dots \otimes H \otimes H}^{n \text{ times}}.$$

This works for any given matrix. When utilising it on an m -qubit gate M_m , one must keep in mind that $M_m^{\otimes n}$ acts on $m \times n$ qubits.

As we observed before, there is an exponential growth in the size of the vectors with the linear increase of qubits. This is also true for matrices, whose size for n qubits is $2^n \times 2^n$. Because of this, when we increase the number of qubits to around 50-75, we start to reach the limits of computation for even world’s best supercomputers when we try to compute these (Y. Zhou, Stoudenmire, and Waintal 2020).

2.5 Circuits

With gates, one is able to create a quantum circuit. A circuit is merely the applications of multiple gates to a set of qubits. For example, one can apply an H gate on the first qubit and a CX gate from the first to the second qubit. This circuit is able to create the Bell state $|\Phi^+\rangle$ mentioned in Section 2.2. The circuit is shown in Figure 2.13.

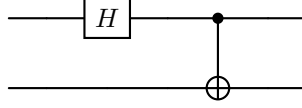


Figure 2.13: A circuit creating the $|\Phi^+\rangle$ Bell state.

The workings of this circuit are simple. All circuits are to be read from left to right, with each qubit on a separate line, numbered from top to bottom. One may assume the qubits be in the $|0\rangle$ state if nothing claims otherwise. This all entails that in the circuit of Figure 2.13, we first apply a Hadamard gate to the first qubit, putting that qubit in an equal superposition. Then, we apply a controlled NOT gate from this superposition to the second qubit. This means that if the value of the first qubit turns out to be 0, the controlled gate does nothing and the second qubit remains as 0, too. If the first qubit turns out to be 1, the controlled gate applies a NOT gate and the second qubit turns into a 1 as well.

Circuits can become more advanced quickly. Some special gates will apply to multiple qubits at the same time. This is visualised in Figure 2.14. For the Bernstein–Vazirani algorithm, there is even an arbitrary number of qubits. How this is visualised can be seen in Figure 2.15. Finally, the *measurement* measures one or multiple qubits, as seen in Figure 2.16.

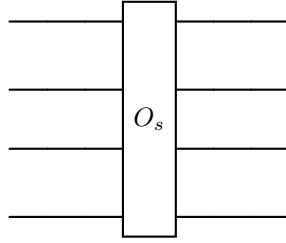


Figure 2.14: A circuit applying a O_s gate to four qubits.

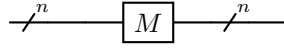


Figure 2.15: A circuit with gate M acting on n qubits.

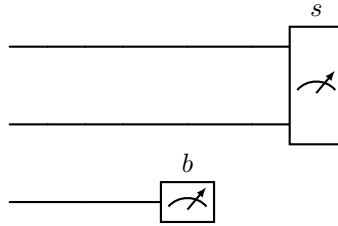


Figure 2.16: A circuit measuring the first two qubits and yielding the outcome $s \in \{0, 1\}^2$, and the third qubit yielding $b \in \{0, 1\}$.

2.6 Algorithms

Finally, we should cover the composition gates to form algorithms. This section describes each of the algorithms formalised in this thesis.

Each of the algorithms I consider make use of something called an *oracle*, denoted with the symbol O_s . This is a special gate with behaviour that is defined by a function f with secret

input string s . The function f usually outputs a single bit. Each application of the oracle should be interpreted as a single access to the corresponding secret input string s through function f .

To consider what an oracle does, I will compare the *classical* oracle to the *quantum* oracle. For a *classical* oracle, the oracle maps any bitstring $t \mapsto f(t)$ for the oracle function f . This function f has access to the secret input string s , and could be any function with input t , such as the inner product between s and t modulo 2. Any call to the oracle counts as a single access to s . For the *quantum* oracle, the oracle maps any qubit state $\forall t : |t\rangle \mapsto (-1)^{f(t)} |t\rangle$. Again, the function f has access to the secret input string s and can be any function with input t .

A simple example of an oracle would be $O_s = X^s$, an oracle taking in a secret input $s \in \{0, 1\}$ and outputting a gate. Its gate is different for when s is 0 or 1: I and X respectively. From this, we can determine the following. Given $|\varphi\rangle = \alpha|0\rangle + \beta|1\rangle$, then

$$O_s |\varphi\rangle = \begin{cases} \alpha|0\rangle + \beta|1\rangle & \text{if } s = 0, \\ \beta|0\rangle + \alpha|1\rangle & \text{if } s = 1 \end{cases}.$$

2.6.1 Deutsch

The Deutsch algorithm is a single-qubit algorithm. It has an oracle with the following specifications:

- **Secret input:** $s \in \{0, 1\}^2$,
- **Oracle function:** $f : \{0, 1\} \mapsto \{0, 1\}$ given by $f(t) = s_t$,
- **Desired output:** $f(0) \oplus f(1)$.

The oracle looks as follows (given the concatenation $s = s_1 s_2$):

$$O_s = \begin{bmatrix} (-1)^{s_0} & 0 \\ 0 & (-1)^{s_1} \end{bmatrix}.$$

The Deutsch algorithm is able to compute the desired value with a single access to the oracle. The quantum circuit for the Deutsch algorithm is given in Figure 2.17.

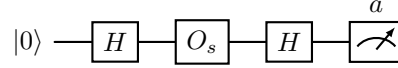


Figure 2.17: The Deutsch algorithm.

The measured value $a \in \{0, 1\} = f(0) \oplus f(1)$. We can prove this by working out the matrix multiplication:

$$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} (-1)^{s_0} & 0 \\ 0 & (-1)^{s_1} \end{bmatrix} \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} (-1)^{s_0} + (-1)^{s_1} & (-1)^{s_0} - (-1)^{s_1} \\ (-1)^{s_0} - (-1)^{s_1} & (-1)^{s_0} + (-1)^{s_1} \end{bmatrix}$$

and applying this matrix to $|0\rangle$:

$$\begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} (-1)^{s_0} + (-1)^{s_1} & (-1)^{s_0} - (-1)^{s_1} \\ (-1)^{s_0} - (-1)^{s_1} & (-1)^{s_0} + (-1)^{s_1} \end{bmatrix} = \begin{bmatrix} (-1)^{s_0} + (-1)^{s_1} & (-1)^{s_0} - (-1)^{s_1} \end{bmatrix}.$$

If s_0 and s_1 are equal, the first value $(-1)^{s_0} + (-1)^{s_1} = \pm 2$ and the latter value $(-1)^{s_0} - (-1)^{s_1} = 0$, and if s_0 and s_1 are not equal, the opposite would be true. Thus, for $s_0 = s_1$, the outcome is $|0\rangle$ and for $s_0 \neq s_1$, the outcome is $|1\rangle$.

2.6.2 Bernstein–Vazirani

The Bernstein–Vazirani algorithm works on a variable number of qubits n . It utilises an oracle with the following specifications:

- **Secret input:** $s \in \{0, 1\}^n$,
- **Oracle function:** $f : \{0, 1\}^n \mapsto \{0, 1\}$ given by $f(t) = t \cdot s \pmod 2$,
- **Desired output:** s .

The oracle for the quantum algorithm thus performs the mapping $|t\rangle \mapsto (-1)^{t \cdot s} |t\rangle$. If the inner product $t \cdot s \pmod 2$ equates to 1, it changes the phase of the standard basis vector. When it is applied on a superposition, it changes the relative phase between the different standard basis states. The Bernstein–Vazirani algorithm provides the desired value s with a single access to the oracle.

We must now define how this oracle O_s is constructed. As mentioned previously, a Z gate flips the phase of a bit, which is exactly what is required here. Based on the index i of each qubit, we apply Z^{s_i} to this qubit. This will flip its phase if s_i is 1, but leave the phase be if it is 0. Its circuit form is depicted in Figure 2.18.

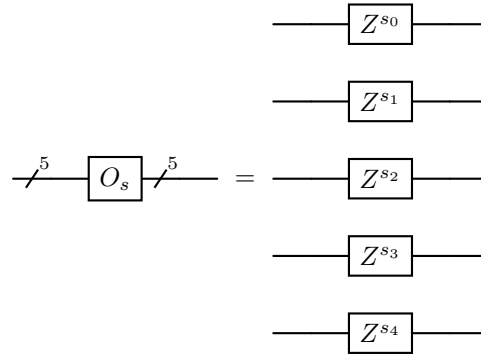


Figure 2.18: The Bernstein–Vazirani oracle circuit for 5 qubits.

With this, we can define the full circuit in Figure 2.19.

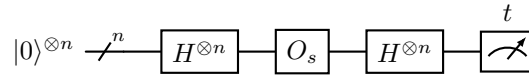


Figure 2.19: The Bernstein–Vazirani algorithm circuit.

In essence, this looks quite similar to the Deutsch algorithm above, but the main changes are the variable number of qubits and the oracle used. As for the measured value of t , it will be equal to s . This is reasonably straight-forward, as there are two cases for any s_i , namely 0 and 1. For 0, the exponent of Z becomes zero, $Z^0 = I$, and the Hadamards cancel out, so nothing happens to the qubit state. If $s_i = 1$, then $H * Z * H = X$ and the qubit state flips. Therefore, we perfectly craft the s state.

2.6.3 One-out-of-Four Grover

The final algorithm to cover is the one-out-of-four Grover algorithm. Its oracle specifications are as follows:

- **Secret input:** $s \in \{0, 1\}^2$,
- **Oracle function:** $f : \{0, 1\}^2 \mapsto \{0, 1\}$ given by $f(t) = \delta_{st}$,
- **Desired output:** s .

Here, the delta function is the Kronecker delta:

$$\delta_{ij} = \begin{cases} 0 & \text{if } i \neq j, \\ 1 & \text{if } i = j. \end{cases}$$

Also note that one can replace the second variable with a numeric value: e.g. δ_{i0} or δ_{i10} .

The one-out-of-four Grover algorithm can find the desired value s with a single access to the oracle.

The oracle gate is simple for this algorithm. It is equivalent to an identity matrix except that on index s , its value is -1 instead of 1:

$$O_s = \begin{bmatrix} (-1)^{\delta_{s00}} & 0 & 0 & 0 \\ 0 & (-1)^{\delta_{s01}} & 0 & 0 \\ 0 & 0 & (-1)^{\delta_{s10}} & 0 \\ 0 & 0 & 0 & (-1)^{\delta_{s11}} \end{bmatrix}.$$

For $s = 11$, this oracle is easy to create: it is equivalent to a controlled Z gate. Moreover, we can fabricate the other oracle circuits with a controlled Z gate. By utilising X gates, we can move where the -1 would be. Placing an X gate on both qubits before the CZ gate will move the -1 to index 0, while removing the bottom X gate moves it to 1, and removing the top gate moves it to 2. See Figure 2.20.

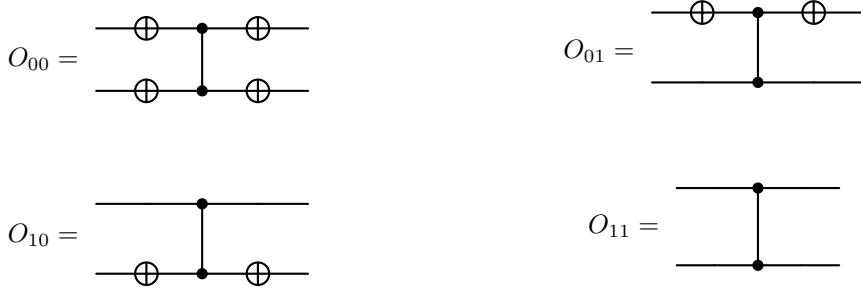


Figure 2.20: The one-out-of-four Grover oracle circuits.

Now, we can create the complete circuit for the one-out-of-four Grover algorithm, as seen in Figure 2.21.

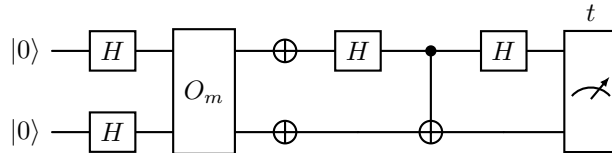


Figure 2.21: The one-out-of-four Grover algorithm circuit with measurement $t \in \{0, 1\}^2$.

The measured value t will again equal s , so $t = s$.

Understanding Lean

As indicated, I will formalise quantum algorithms in Lean 4. To ease the reading of this paper, I will explain the basics and the components of the Lean programming language and its mathematics library `mathlib` that we will require to write the formalisations in Lean. *Please note that this is not a Lean tutorial: this is only meant to give a basic idea of Lean.*

Lean is a theorem prover and a programming language, but we will stay close to its theorem prover aspect. In Section 3.1, we will cover its typing system, theorems and tactics. I will continue to explain in Section 3.2 how matrices are defined in `mathlib` and they can be defined and used. I will also cover important matrix-related theorems from `mathlib` here. Finally, I will go over induction and recursion, which will allow variable-sized algorithms to be formalised. This will be explained in Section 3.3.

If you are looking for coverage on the complete basics of Lean and how to utilise it as a language, I would like to suggest to take a look at the many resources written and made by the Lean community. There is even entire interactive books that take you through learning Lean. I found the [*Learning Lean 4*](#) page on the Lean Community website a useful hub for resources.

3.1 The Basics of Lean

Lean has a very strict typing system. Every variable, every constant and even every *type* has a type. To explain a bit about these types, let us consider a simple variable.

```
def two := 2
```

With this statement, Lean is already doing some things under the hood. Our variable `two` has received the implicit type `Nat`, or $\mathbb{N}$. We can make this type definition explicit, too.

```
def two : N := 2
```

If we go back to having no type, we can also make the value -2 . Lean immediately recognises that it needs to change the type to an `Int` or $\mathbb{Z}$.

```
def two := -2
#check two -- two : Z
```

Now, we can also define a different type for the variable `two`. Perhaps we want it to be a rational number or a complex number. Logically, we assign the types $\mathbb{Q}$ or $\mathbb{C}$. However, since these are `mathlib` types, we need to import the file that defines these types.

```
import Mathlib.Data.Rat.Init
import Mathlib.Data.Complex.Basic

def two : Q := 2
def two : C := 2
```

Lean automatically handles the conversion between our written 2 and the rational and complex number types. For example, the complex number type internally requires two real numbers $p \in \mathbb{R}$ and $q \in \mathbb{R}$ for its $p + qi$ representation. We can actually see the value of a variable using the `#eval` command. Note that in the next code, `--` signifies a comment, in this case the output of `#eval`. I will leave out previous contexts and imports for brevity from now on.

```
def two : C := 2
#eval two
-- Real.ofCauchy (sorry /- 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, ... -/) +
-- Real.ofCauchy (sorry /- 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ... -/)*I
```

The Cauchy sequence is how `mathlib` is defining the real numbers. I found for simple definitions such as this (`def two : C := 2`), one can assume the value will not change further on in the sequence. So the above is readable as $2 + 0i$, as expected. Note that Lean, or `mathlib` to be specific, defines its imaginary number as `I`.

Now for something more advanced, functions. We can define an `addTwo` function like so:

```
def addTwo (m : N) : Q := 2 + m
```

This takes in a *natural* number and gives back a rational number. Leaving `m` untyped would result in an error as Lean could not derive the type it should have from its context. We can also make `m` have a type of `Q` to make this function specifically for rational numbers.

In general, a definition adheres to the following syntax:

```
def definitionName (argumentA : ArgumentAType) ... : Type := Definition
```

Lean also brings another command to the table: `theorem`. Let us have a look at the syntax.

```
theorem theoremName (propertyA : PropertyAType) ... : Goal := Proof
```

In syntax, `def` and `theorem` are technically equivalent. It is entirely possible to write everything with `def` only. However, Lean makes an important distinction between the two. A `theorem` is marked as *irreducible*, meaning it does not get unfolded. A definition is not marked as such, which means Lean processes the definition itself instead of only applying the equivalence. It is very important to utilise the `theorem` command over the `def` command, because Lean will do much less work when working with theorems. It will also be able to check the proofs of theorems in parallel, as the theorems that another theorem relies on will be assumed true.

Theorems need to be proven, and to do this, Lean generates goals. The first goal is the one defined in the theorem, which can be rewritten until it is equivalent to another theorem, or solved by tactics. I will cover the tactics `rewrite` (abbreviated to `rw`), `rfl`, `exact`, `simp`, `norm_num`, and `abbrev` along with their searching variants for the tactics that have these.

3.1.1 `rw`

To explain the `rw` tactic, let us consider a simple theorem that works on a natural number m and states $m + m = m * 2$.

```
theorem add_self_eq_mul_two (m : N) : m + m = m * 2 := by
-- 1 goal
-- m : N
-- ⊢ m + m = m * 2
```

Proving this theorem is relatively simple. We want to rewrite either side to be equivalent to the other side. So we either need to turn $m + m$ into $m * 2$, or vice versa. To use `rw`, we need theorems to utilise or *rewrite*. For this problem, we can use the theorem `Nat.mul_two`, which is taken from the `Nat` namespace. When rewriting, Lean will automatically attempt to find any instance that corresponds to the theorem given.

```
theorem add_self_eq_mul_two (m : N) : m + m = m * 2 := by
  rw [Nat.mul_two] -- m + m = m + m
-- No goals
```

Lean substitutes $m * 2$ for $m + m$ and already recognises that the two sides are identical and closes the goal. We can also apply the theorem the other way around, by using an `←` symbol.

```

theorem add_self_eq_mul_two (m : N) : m + m = m * 2 := by
  rw [← Nat.mul_two] -- m * 2 = m * 2
  -- No goals

```

In this specific case, going either direction is fine as both sides match to something. However, most of the time, we will find that going one way is not possible. For example, we can't apply `Nat.mul_one` because we do not have an instance that is multiplied by 1 (e.g. $m * 1$). However, it is possible to apply it the other way around:

```

theorem add_self_eq_mul_two (m : N) : m + m = m * 2 := by
  rw [← Nat.mul_one]
  -- tactic 'rewrite' failed, pattern is a metavariable
  -- ?n
  -- from equation
  -- ?n = ?n * 1
  -- m : N
  -- ⊢ m + m = m * 2

```

This code fails, however. Lean is telling us that it does not know what `?n` is supposed to be. We basically did not tell it what to multiply with 1. So, we need to make sure that we do this:

```

theorem add_self_eq_mul_two (m : N) : m + m = m * 2 := by
  rw [← Nat.mul_one m]
  -- 1 goal
  -- m : N
  -- ⊢ m * 1 + m * 1 = m * 1 * 2

```

You may look at this and wonder how we can only multiply the second m with 1. This is what `nth_rw` is for:

```

theorem add_self_eq_mul_two (m : N) : m + m = m * 2 := by
  nth_rw 2 [← Nat.mul_one m]
  -- 1 goal
  -- m : N
  -- ⊢ m + m * 1 = m * 2

```

That covers the basic use-cases of `rw`.

3.1.2 `rfl` and `exact`

Sometimes, a theorem is nearly complete, or is identical on both sides but Lean did not mark the goal as complete yet. In this case, we have `rfl`, which stands for *reflexive*. For example, we can immediately prove $1 + 1 = 2$ using `rfl`:

```

theorem one_plus_one_eq_two : 1 + 1 = 2 := by
  rfl
  -- No goals

```

`rfl` is technically a shorthand for `exact rfl`. The commands `exact` can be used to assert that a certain goal is identical to a given expression. For a theorem stating $m * 2 = m + m$, we can also use `exact` to solve this proof:

```

theorem mul_two_eq_add_self (m : N) : m * 2 = m + m := by
  exact Nat.mul_two m
  -- No goals

```

Here, you may notice we are stating that `Nat.mul_two m` is equivalent to our goal, which therefore closes the goal. If we swap around the $m * 2$ and $m + m$, we get an error, as it does not recognise the expressions to be identical. We can fix this by using the fact that equality is symmetrical, or in Lean: `Eq.symm`.

```

theorem mul_two_eq_add_self (m : ℕ) : m + m = m * 2 := by
  exact Nat.mul_two m
  -- type mismatch
  --   Nat.mul_two m
  -- has type
  --   m * 2 = m + m : Prop
  -- but is expected to have type
  --   m + m = m * 2 : Prop

theorem mul_two_eq_add_self (m : ℕ) : m + m = m * 2 := by
  exact Eq.symm (Nat.mul_two m)
  -- No goals

```

3.1.3 simp

The `simp` tactic stands for *simplifier* and utilises the definitions and theorems that it has been given to attempt to simplify a goal. You can give any lemmas or hypotheses to it, and it will attempt to apply them to come closer to closing your goal. Let's take a theorem proving $j^2 = (j + 1)(j - 1) + 1$.

```

theorem one_plus_one_eq_two (j : ℤ) : j ^ 2 = (j + 1) * (j - 1) + 1 := by
  rw [mul_sub] -- j ^ 2 = (j + 1) * j - (j + 1) * 1 + 1
  rw [add_mul] -- j ^ 2 = j * j + 1 * j - (j + 1) * 1 + 1
  rw [mul_one] -- j ^ 2 = j * j + 1 * j - (j + 1) + 1
  rw [← sub_sub] -- j ^ 2 = j * j + 1 * j - j - 1 + 1
  rw [one_mul] -- j ^ 2 = j * j + j - j - 1 + 1
  rw [add_sub_cancel_right] -- j ^ 2 = j * j - 1 + 1
  rw [sub_add_cancel] -- j ^ 2 = j * j
  rw [pow_two] -- No goals

```

This code proves the theorem using only rewrites. You can see the progress of the goal in the comments of the code. We can use `simp` to help solve this for us. Sadly, it doesn't work when only using `simp`.

```

theorem one_plus_one_eq_two (j : ℤ) : j ^ 2 = (j + 1) * (j - 1) + 1 := by
  simp -- simp made no progress

theorem one_plus_one_eq_two (j : ℤ) : j ^ 2 = (j + 1) * (j - 1) + 1 := by
  rw [mul_sub, add_mul, mul_one, ← sub_sub]
  -- j ^ 2 = j * j + 1 * j - j - 1 + 1
  simp -- j ^ 2 = j * j
  rw [pow_two] -- No goals

theorem one_plus_one_eq_two (j : ℤ) : j ^ 2 = (j + 1) * (j - 1) + 1 := by
  rw [mul_sub, add_mul, mul_one, ← sub_sub]
  -- j ^ 2 = j * j + 1 * j - j - 1 + 1
  simp [pow_two]
  -- No goals

```

The first bit of code shows that Lean doesn't know how to simplify from the first expression. So in the second example, we rewrite to a suitable state for `simp` to take over. Then, `simp` does its magic and ends up at $j^2 = j \cdot j$. We can either finish the proof with a rewrite, or tell `simp` to also allow the use of `pow_two` in its simplification.

To make Lean utilise more theorems for `simp`, we can tag theorems with `@[simp]`. Theorems marked as such will automatically get utilised by `simp`.

Before utilising `simp` in every place and let it do all the work, I suggest reading Subsections 3.1.5 and 3.1.6. These will explain how to find suitable theorems to use in your proof and explain why it is better to apply rewrites than to use `simp`.

3.1.4 norm_num

Numerical expressions can sometimes be normalised. To Lean, `2` and `1 + 1` are not equivalent in a typed sense, so a goal which looks like `1 + 1 = 2` is not automatically closed. Usually, this goal can be closed with `rf1`, but if two matrices were compared with `1 + 1` and `2` at a given index of each matrix, it would not suffice.

The `norm_num` command, compared to `rw` and `simp`, does handle this, as it normalises the two numerical expressions to be identical. This can make solving equations quite a bit simpler, certainly when defining equalities of certain circuits such as $H \cdot X \cdot H = Z$.

```
def H := !![1, 1; 1, -1]
def X := !![0, 1; 1, 0]
def Z := !![1, 0; 0, -1]

theorem HXHeqZ : H * X * H = 2 * Z := by
  simp [H, X, Z]
  -- 1 goal
  -- of ![0, 1 + 1], ![1 + 1, 0]] = of ![0, 2], ![2, 0]]

theorem HXHeqZ : H * X * H = 2 * Z := by
  norm_num [H, X, Z]
  -- No goals
```

In essence, `norm_num` is like `simp` but a bit more powerful. It is possible to give `norm_num` extra theorems to apply, just like with `simp`.

3.1.5 Finding Suitable Theorems

With the great amount of theorems that Lean and `mathlib` provide, it is sometimes difficult to find the right theorems to apply. There are a few ways of *searching* through the theorem library quite easily. I'd first like to shine light on the `?` command suffix.

It is possible to use a `?` after certain tactics. You can use `rw?`, `exact?` and `simp?` to let Lean find suitable rewrites or theorems to apply. It will generate a list with possibilities, which I found to usually be in order of relevance or rather, simplicity. When we take the previous theorem for $j^2 = (j+1)(j-1) + 1$, we can see how each of these work.

```
theorem one_plus_one_eq_two (j : ℤ) : j ^ 2 = (j + 1) * (j - 1) + 1 := by
  rw?
  -- Try this: rw [Int.mul_sub]
  -- -- j ^ 2 = (j + 1) * j - (j + 1) * 1 + 1
```

The rewrite command with a `?` gave twenty suggestions for how to rewrite the goal into something else. The first suggestion suggested to use `Int.mul_sub` to rewrite for $i \cdot (j - k) = i \cdot j - i \cdot k$.

For `exact`, we can apply that right after a `simp`:

```
theorem one_plus_one_eq_two' (j : ℤ) : j ^ 2 = (j + 1) * (j - 1) + 1 := by
  rw [mul_sub, add_mul, mul_one, ← sub_sub]
  simp
  exact? -- Try this: exact pow_two j
```

It suggests that `pow_two j` is identical to our goal.

For `simp`, things are a bit different. Whenever I use `simp`, I add a `?`. It generates the list of theorems that `simp` utilises under the hood. This can then be added to `simp` so it *only* uses those theorems and therefore becomes more efficient. This optimises the proof by restricting `simp` to only a handful of theorems.

```
theorem one_plus_one_eq_two' (j : ℤ) : j ^ 2 = (j + 1) * (j - 1) + 1 := by
  rw [mul_sub, add_mul, mul_one, ← sub_sub]
  simp?
  -- Try this: simp only [one_mul, add_sub_cancel_right, sub_add_cancel]
  -- j ^ 2 = j * j
```

Note that it did not suggest using `pow_two`, as `simp` does not know about this theorem. We can tell `simp` to use it in its proof: `simp? [pow_two]`. This would close the goal immediately.

3.1.6 Proof Efficiency

In general, Lean is relatively fast at checking proofs. However, the over-usage of powerful commands such as `simp` and `norm_num` can lead to a slowdown, because their execution is not bound to a single theorem like `rw` is.

I suggest when using `simp` to always add a `?` and let Lean tell you exactly what `simp` needs so it doesn't need to compute it again. Sometimes, `simp` applies relatively simple rules that can also be rewritten. This is again faster. For example, one bit of code that I had written used `norm_num` where I could have rewritten two theorems and also be done. Always prefer rewrites, then `simp` only, and then `norm_num`.

Since `norm_num` is a slightly stronger version of `simp`, always check if `simp?` finds a solution and use that instead.

In general, Lean does a lot of work for you, but utilising the tools it gives you such as `rw?` and `simp?` can give you the theorems that Lean is using under the hood. More explicit rewrites will always be faster than Lean doing the work for you.

3.1.7 abbrev

Finally, I want to talk about `abbrev`. This command is identical to a definition, except that Lean knows it is *reducible*. As I indicated with theorems, it is best to keep definitions as *irreducible* as possible, so we do not unfold definitions and cause Lean to do more work. So why use this?

Certain type definitions tell Lean to use a certain set of methods, e.g. to cast an integer to a complex number. When using `def` to define a new `Nat` type, for example, Lean loses this knowledge on what to use. However, `abbrev` is replaced, so the actual type is used instead of the new type. This will prove useful when abbreviating `Matrix` definitions later on. The syntax is identical to `def`.

3.2 Matrices

Matrices are a big part of what I am utilising in this thesis. We define the initial building blocks, the gates, as matrices, and build up from there using functions such as the Kronecker product. We will first consider the syntax of the `Matrix` type in `mathlib`.

3.2.1 Matrix

```
/-- `Matrix m n R` is the type of matrices with entries in `R`, whose rows are indexed
by `m` and whose columns are indexed by `n`. -/
def Matrix (m : Type u) (n : Type u') (α : Type v) : Type max u u' v :=
  m → n → α
```

This code looks very complicated, but in fact, with a bit of intuition, we can understand how to utilise `Matrix`. We need three arguments to define a matrix, two dimensions m and n , as well as the type of the value, α . The m and n require a finite type, indicated by `Type u` and `Type u'`. This can be indicated with a finite set, written as `Fin o`, where o is a natural number, and the finite sets contains all natural numbers smaller than o . As the types of m and n are different, we can logically provide different sizes for the rows and the columns. The α defines the type of matrix entries, such as a natural number or a complex number. Finally, the output type is a little strange. It is written as the *largest universe of* u , u' and v and becomes a function with two inputs with types m and n , and outputs a value with type α . One can read this definition as a function that receives two indices and outputs the value at those indices.

Defining a matrix goes as follows.

```
import Mathlib.Data.Matrix.Basic
import Mathlib.Data.Matrix.Notation

open Matrix

def H : Matrix (Fin 2) (Fin 2) ℤ := !![1, 1; 1, -1]
```

We import the `Matrix` type, and the notation shorthand through the imports, and then we open `Matrix` so Lean includes the definitions in the `Matrix` files so we can use them. Then we define `H`, our Hadamard matrix, in its non-normalised form. It is a 2×2 matrix, so we provide it with `Fin 2` for both dimensions, and assign the integer type for the values of the entries. Then we use the `!![]` notation to define the matrix itself. The comma-separated numbers form a row, and

each semicolon marks the end and beginning of two rows. So, `!![1, 1; 1, -1]` and `!![i, j; k, l]` define the following matrices:

$$\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}; \quad \begin{bmatrix} i & j \\ k & l \end{bmatrix}.$$

3.2.2 Operators

The usual operations can be performed on matrices, such as addition, subtraction and multiplication. Scalar multiplication is also possible. However, since Lean casts numerical values to multiples of the identity matrix, one must realise scalar addition and subtraction are not a thing.

```
#eval H + H -- !![2, 2; 2, -2]
#eval H - H -- !![0, 0; 0, 0]
#eval H * H -- !![2, 0; 0, 2]
#eval 2 * H -- !![2, 2; 2, -2]
#eval 1 + H -- !![2, 1; 1, 0]
#eval 1 - H -- !![0, -1; -1, -2]
```

The `norm_num` command comes in handy at times when applying operators. As indicated before, `simp` or `rw` cannot prove `!![0, 1 + 1; 1 + 1, 0] = !![0, 2; 2, 0]` in a single command, while `norm_num` can. Note that `#eval` also normalises, like `norm_num`.

3.2.3 Kronecker Product

The Kronecker product also comes built-in, in `mathlib`. It utilises the $\otimes_k$ symbol.

```
import Mathlib.Data.Matrix.Kronecker

open Kronecker

def H : Matrix (Fin 2) (Fin 2) ℤ := !![1, 1; 1, -1]
def HH := H ⊗_k H

#check HH -- HH : Matrix (Fin 2 × Fin 2) (Fin 2 × Fin 2) ℤ
```

3.2.4 reindex

Note how the Kronecker product takes the direct product of the dimension types and does not simply define it as `Fin 4`. In its current form, we cannot `#eval` the `HH` matrix, however it is possible to *reindex* the matrix to overcome this issue. Reindexing applies an equivalence, in this case $(\text{Fin } 2 \times \text{Fin } 2) \simeq \text{Fin } 4$, to the dimensions of the matrix. Note that `mathlib` defines $\simeq$ as a shorthand for `Equiv`.

```
def Fin2x2_4 : (Fin 2 × Fin 2) ≃ Fin 4 := by
  ...

#eval reindex Fin2x2_4 Fin2x2_4 HH
-- !![1, 1, 1, 1; 1, -1, 1, -1; 1, 1, -1, -1; 1, -1, -1, 1]
```

As can be seen, we get the following matrix, which we expect:

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}.$$

Reindexing is a requirement due to the strict dependent typing system that Lean utilises. I will discuss this in more detail in Section 4.5 of Chapter 4. As matrix sizes are clear for us, I will gloss over the reindexing that is being done to remain concise and clear. Possible solutions to this problem are discussed in the Further Work chapter, Chapter 5.

3.2.5 Equality and ext

It should be possible to determine the equality of two matrices. Sometimes, it is not as straightforward as stating `norm_num` and being done with it. Thus, we have the `ext` command. The corresponding `mathlib` theorem states that two matrices are equal if each of their entries are equal.

```
theorem ext : (∀ i j, M i j = N i j) → M = N
```

The Lean code above can be read as *for each i and j , if $M i j = N i j$, then $M = N$.*

Its eponymous command can be applied to a goal stating the equality of two matrices. Stating `ext i j` turns the goal into one where the two matrices are entry-wise identical. With the `fin_cases` command, we can then go through each row and column and verify the equality. The `<;>` indicates that all latter commands should be applied to every goal. So when `fin_cases i` creates two goals for $i = 0$ and $i = 1$, we apply `fin_cases j` to both options, creating four goals in total, instead of only applying `fin_cases i` to the goal with $i = 0$.

```
theorem one_fin_two : (1 : Matrix (Fin 2) (Fin 2) ℕ) = !![1, 0; 0, 1] := by
  -- 1 = !![1, 0; 0, 1]
  ext i j
  -- 1 i j = !![1, 0; 0, 1] i j
  fin_cases i <;> fin_cases j <;> rfl
  -- No goals
```

3.3 Recursion and Induction

Finally, we have recursion. The `tensor_power` as explained in Chapter 2 can be defined using a recursive definition. To do this, we define a base case and a successive case, in this case written as $(n' + 1)$ (one can also simply write $(n + 1)$ and reuse the variable name). Again, a simplified version of the code is shown, and code output in the comments is simplified.

```
def tensor_power (n : ℕ) (M : Matrix (Fin 2) (Fin 2) ℕ)
  : Matrix (Fin (2 ^ n)) (Fin (2 ^ n)) ℕ :=
  match n with
  | 0 => !![1]
  | (n' + 1) => tensor_power n' M ⊗k M
```

This definition simply applies the tensor power recursively until $n = 0$, when it applies a Kronecker product with `!![1]`, which is an identity operation. As shown, recursion can be done using the `match <variable> with` syntax, and defining cases for the variable. It is also possible to define different cases for 1, 2 or any arbitrary n . When defining an $(n' + 1)$, for example, the value will be determined from the equation $n = n' + 1$. So in the above, each recursion step makes n' one smaller than before.

To go with recursion, we have induction. In places where n can tell us something about $n + 1$, we can use the `induction` command. It splits the goal into two goals, the base case of the theorem, where $n = 0$, and the successor case, where n is replaced with $n + 1$ and a proof of the original goal can be used. In this way, it is very similar to the recursion we just defined.

Let us consider a simplified version of the `tensor_power_mul` theorem that we will define in Lean in Chapter 4. Its inputs are an arbitrary n and two matrices M and N . The `tensor_power` definition further below applies the n -fold Kronecker product to the given matrix. This theorem states that the tensor power of $M \cdot N$ is equivalent to multiplying the tensor powers of M and N :

$$(M \cdot N)^{\otimes n} \simeq M^{\otimes n} \cdot N^{\otimes n}.$$

Induction is initiated using the `induction <variable> with` syntax. Then, two cases need to be proven: the zero case and the `succ` or successive case. The zero case, or base case, replaces the variable, in this case n , with 0. The successive case is indicated with the `succ <new-variable> <hypothesis>` syntax. It provides the original goal as hypothesis, and changes the goal to contain $n + 1$ where there used to be n . I found naming the new variable

identically to before the most intuitive. The rest is all logical, as it is simple induction. When the successive case is proven, Lean marks the theorem as proven.

The code output in the comments of the code is altered for clarity and readability.

```

theorem tensor_power_mul { n : ℕ } (M N : Matrix (Fin 2) (Fin 2) ℕ)
  : tensor_power n (M * N) = tensor_power n M * tensor_power n N := by
  induction n with
  | zero => simp
  | succ n base_case =>
    rw [tensor_power, tensor_power, tensor_power]
    -- tensor_power n (M * N) @_k (M * N) =
    -- (tensor_power n M) @_k M * (tensor_power n N @_k N)

    rw [← mul_kronecker_mul]
    -- tensor_power n (M * N) @_k (M * N) =
    -- (tensor_power n M * tensor_power n N) @_k (M * N)

    rw [base_case]
    -- No goals

    -- (The final rw turned the goal into this)
    -- tensor_power n (M * N) @_k (M * N) =
    -- tensor_power n (M * N) @_k (M * N)

```


Creating Quantum Lean

With the prerequisite information on all relevant topics explained, we can put this knowledge together into a foundation as a Lean package – Quantum Lean. This package will implement the quantum theory as discussed in Chapter 2 on top of Lean, using the features as discussed in Chapter 3.

I will go through the main parts of Quantum Lean in order of definition, starting with the core types. Here, I will cover the custom types defined in Quantum Lean and what their purpose is.

In the theorem section, I will cover the theorems defined for the rewriting of algorithms. This includes proofs on the Kronecker product, tensor powers, and reindexing.

After that, I will go over the definitions of the gates such as the Hadamard and Pauli gates. This is the first implementation in Quantum Lean. I will also cover the formalised theorems surrounding them.

Then, I will cover the formalisation of algorithms. After that, I will lay out the difficulties that I have come across in the making of this framework. I will indicate how these difficulties are currently handled. I will also link to any relevant further work that may solve these difficulties.

Finally, I will talk about the documentation of Quantum Lean.

4.1 Core Types

Quantum Lean has a set of definitions for the size of matrices in terms of the number of qubits, a definition for qubits themselves and for gates applied on qubits. These definitions abstract away the properties that are not necessary for Quantum Lean, such as non-square matrices.

The effect of this abstraction is that writing proofs becomes more straightforward, and theorems can be simpler as we only need to consider the subset of the type, such as the square matrices versus any matrices, or 2^n sized vectors describing qubit states versus any length vectors.

```
abbrev QCount (n : ℕ) := Fin (2 ^ n)
```

The above definition of `QCount` defines the size of the vector describing n qubits. With this abstraction, we can define the equivalences between different `QCount` usages. These equivalences can be applied in the dimensions of our qubit and gate definitions, rewriting them when necessary as Lean does not handle this on its own.

For example, a proof for $(\text{QCount } m \times \text{QCount } n) \simeq \text{QCount } (m + n)$ can be given as follows.

```
def QCount_mul_QCount : (QCount m × QCount n) ≃ QCount (m + n) := by
  rw [QCount, QCount, QCount, pow_add]
  -- Fin (2 ^ m) × Fin (2 ^ n) ≃ Fin (2 ^ m * 2 ^ n)

  exact @finProdFinEquiv (2 ^ m) (2 ^ n)
```

After rewriting the abbreviation `QCount`, we can rewrite 2^{m+n} into $2^m \cdot 2^n$ with the `pow_add` theorem, and then prove the equivalence using the `finProdFinEquiv` or finite number product equivalence theorem. This theorem shows that $\text{Fin } a \times \text{Fin } b \simeq \text{Fin } (a * b)$. With this abbreviation, the power of two equivalences are hidden so needless complexity is hidden. This makes it easier to see how many qubits a gate affects.

Next up, I will be going through the gate and qubit state definitions. There are a couple, in two categories. The normal abbreviations meant to be used throughout the codebase, and the primed (marked with a `'`) abbreviations meant as a more `mathlib`-like definition.

`nGate` and `nQubit` are the most simple types, being a gate acting on n qubits, and a vector describing an n qubit state.

```
abbrev nGate (n : ℕ) := nGate' n n
abbrev nQubit (n : ℕ) := nGate' 0 n
```

Its definitions are defined in `nGate'`, which allows for setting different sizes for either dimension of the matrix. This generalisation makes it possible to apply the definition to qubit states without having to create new theorems and interlinking them with existing theorems. This would have required twice the theorems on their own and new ones to prove that e.g. the tensor power of qubits multiplied with the tensor power of gates is also possible. The generalisation proves it is possible without any extra code. As indicated, these primed abbreviations are not supposed to be used outside of further definitions. The only two possible matrices definable in Quantum Lean without primed abbreviations are square ones (through `nGate`) and vectors for qubit states, as they are defined above.

As for `nGate'`, it utilises the `QCount` definition from before, and creates a matrix with complex numbers as its entries. This definition is an intermediary representation to write more general theorems. In general, gates should be defined with `nGate`.

```
abbrev nGate' (n n' : ℕ) := Matrix (QCount n) (QCount n') ℂ
```

Furthermore, there are two more definitions for matrices. These are also intermediary representations, to allow the tensor power definition to function with little hassle. These definitions are `mnGate'` and `mnGate''`. The former is again a square variant of the latter. Both are only supposed to be used internally in theorems and not in proofs.

```
abbrev mnGate' (m n : ℕ) := mnGate'' m m n n
```

```
abbrev mnGate'' (m m' n n' : ℕ) :=
  Matrix (QCount m × QCount n) (QCount m' × QCount n') ℂ
```

These definitions are required to reindex the output of the Kronecker product. Given the Kronecker product between two matrices, one for m qubits and one for n qubits (ergo: an `nGate m` and an `nGate n`), their resulting type doesn't become `nGate (m + n)` but `Matrix (QCount m × QCount n) (QCount m × QCount n) ℂ`. This is a problem as this type is not equivalent to the types defined above, which means our theorems cannot be applied to them anymore.

Thus, `mnGate'` and `mnGate''` make the definitions for reindexing more concise and legible.

4.2 Theorems

4.2.1 Kronecker Product

The set of theorems pertaining to the Kronecker product in `mathlib` is substantive, but was lacking in a set of theorems on how the Kronecker product handled `natCasts`, or natural numbers cast into a multiple of the identity matrix.

The Quantum Lean library offers five theorems on the Kronecker product.

- natCast_kronecker
- kronecker_natCast
- kronecker_natCast_natCast
- kronecker_natOne
- kronecker_matOne

The first two theorems prove that given a diagonal matrix (`natCast`) D with the natural number d on its diagonal, and an arbitrary matrix M , that $D \otimes M = d \cdot (\mathbb{1} \otimes M)$ and $M \otimes D = d \cdot (M \otimes \mathbb{1})$, where $\mathbb{1}$ is the identity matrix with the same size as D .

```
theorem natCast_kronecker {A B : Type} [DecidableEq A] [DecidableEq B]
  (m : ℕ) (M : Matrix B B C) :
  (m : Matrix A A C) ⊗k M = m • ((1 : Matrix A A C) ⊗k M) := by
  rw [← nsmul_one, smul_kronecker]
```

This code is more reminiscent of `mathlib` code, as it is fairly abstract in its type definitions for dimensions A and B . In essence, this is saying that we have two separately sized matrices, one of size $A \times A$ and $B \times B$. The `[DecidableEq A]` and `[DecidableEq B]` tell us that for A and B , we have decidable equality, that is to say that we can always compare them and see if they are equal.

With these more abstract theorems, we have broader theorems that can be applied to a larger set of proofs. While it won't be more helpful to Quantum Lean, it may find a use within `mathlib`.

The proof is relatively straight-forward, as we separate the value m from the identity matrix with the first replace, and then we apply the `smul_kronecker` theorem, which shows $D \otimes M = m \cdot (\mathbb{1} \otimes M)$, where D is the m th multiple of the identity matrix.

The third theorem shows that a Kronecker product of two `natCasts` is equivalent to the multiple of an identity matrix: $M_1 \otimes M_2 = m_1 \cdot m_2 \cdot \mathbb{1}$ where M_i is the m_i th multiple of the identity matrix, and $\mathbb{1}$ has the same dimensions as the $M_1 \otimes M_2$ matrix.

```
theorem kronecker_natCast_natCast {A B : Type} [DecidableEq A] [DecidableEq B]
  (m1 m2 : ℕ) : (m1 : Matrix A A C) ⊗k (m2 : Matrix B B C) =
    ((m1 * m2 : ℕ) : Matrix (A × B) (A × B) C) := by
  rw [natCast_kronecker, kronecker_natCast, one_kronecker_one, smul_smul, nsmul_one]
```

As can be seen, the `natCast_kronecker` and `kronecker_natCast` are applied in this instance. Then, we apply the `one_kronecker_one` theorem to turn $\mathbb{1} \otimes \mathbb{1}$ into a larger identity matrix. Finally, we apply a couple of theorems to turn the resulting $m_1 \cdot m_2 \cdot \mathbb{1} = m_1 \cdot m_2$ into an equality, so Lean closes the goal.

The last two theorems in the above list are more specific and prove that utilising the Kronecker product on a natural number 1 or a `!![1]` matrix and an arbitrary matrix M , the result is equivalent to M .

```
theorem kronecker_matOne {A B : Type} [DecidableEq A] [DecidableEq B]
  (M : Matrix A B C) : !![1] ⊗k M =
    Matrix.reindex (Equiv.uniqueProd _ _).symm (Equiv.uniqueProd _ _).symm M := by
  rw [← @kronecker_natOne] -- !![1] ⊗k M = (1 : Matrix (Fin 1) (Fin 1) C) ⊗k M
  rw [one_eq] -- No goals
```

We prove `kronecker_natOne` by applying `kronecker_matOne` in reverse, and then applying the theorem `one_eq` which states `!![1] = (1 : Matrix (Fin 1) (Fin 1) C)`.

4.2.2 Tensor Power

The tensor power provides a chain of Kronecker products with a given gate. In Quantum Lean, I first defined `nfold_tensor_product`, which is a little more advanced than that. It is possible to give it a list of gates that it needs to create the n -fold tensor product with. For this, we have the following definition.

```
variable {m m' n n' : ℕ}

def nfold_tensor_product (n : ℕ) (gates : ℕ -> nGate' m m') : nGate' (m * n) (m' * n) :=
  match n with
  | 0 => !![1]
  | (n + 1) => reindex₃ (nfold_tensor_product n gates ⊗ₖ gates (n + 1))
```

First of all, I want to note that the `variable` command adds the subsequent variables to each of the definitions and theorems in a given section, the tensor power section, in this case. Since they are defined in `{ }` brackets, Lean will infer them. Not all variables there will be used in each theorem.

The `gates` function specifies which gate to place on each qubit. Because of this function, the `nfold_tensor_product` definition in Quantum Lean is more powerful than a tensor power, as it can create the n -fold tensor product of different gates, instead of only a single one.

Also notice that different sizes m and m' can be given for different dimensions, this is so the tensor power can apply to both qubit states and gates. Finally, the last line contains a custom `reindex₃` statement, which will be explained in more detail in Subsection 4.2.3.

The rest of the workings of the `nfold_tensor_product` function are simple: we apply a Kronecker product to the n -fold tensor product of the $n - 1$ gates and the last gate supplied via `gates`. The `nfold_tensor_product` thus becomes equivalent to the circuit as displayed in Figure 4.1.

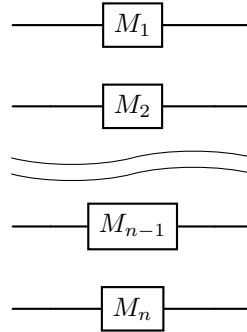


Figure 4.1: Quantum circuit of the n -fold tensor product of the gates $M_1, \dots, M_n$, supplied through the `gates` function, with gates M_3 to M_{n-2} not shown.

Then, there is the definition of the tensor power as defined in Chapter 2.

```
abbrev tensor_power (n : ℕ) (M : nGate' m m') := nfold_tensor_product n (fun _ => M)
```

As can be seen, it requests the same n to define the height of the tensor power, but instead requests a single gate M . It then provides the `nfold_tensor_product` definition with the same n and a function that always provides M . This is a simple shorthand which practically alleviates the writing of `(fun _ => )` all the time. This abbreviation is equivalent to the circuit drawn in Figure 4.2.

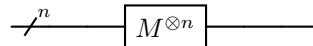


Figure 4.2: A circuit with gate M acting on each of the n qubits individually, as `tensor_power` does.

Finally, the last abbreviation regarding the n -fold tensor product.

```
abbrev apply_at (n i : ℕ) (M : nGate m) :=
  nfold_tensor_product n (fun j => if i == j then M else 1)
```

This definition applies a given single-qubit gate M on a specific qubit. This is useful for applying, for example, a Hadamard or X gate on a specific qubit while leaving other gates unaffected. Its circuit should be clear, as it is a single gate applied to the i -th qubit out of n .

There are eight theorems currently in Quantum Lean surrounding the n -fold tensor product and the tensor power. These are the following:

- `nfold_tensor_product_zero`
- `nfold_tensor_product_mul`
- `nfold_tensor_product_of_one` (and a tensor power variant)
- `nfold_tensor_product_of_natCast` (and a tensor power variant)
- `nfold_tensor_product_smul` (and a tensor power variant)

Note that because of the definition of the tensor power as an n -fold tensor product of the same gate, the n -fold tensor product theorems also apply, but three variants have been written especially for the tensor power to make rewrites simpler.

The first theorem simply shows that `nfold_tensor_product 0 M = !![1]`.

The second theorem shows that $(\bigotimes_{i=1}^n M_i) \cdot (\bigotimes_{i=1}^n N_i) = \bigotimes_{i=1}^n (M_i \cdot N_i)$, which means that we can rewrite the product of two n -fold tensor products as a single n -fold tensor product.

```
theorem nfold_tensor_product_mul (M : ℕ -> nGate' m m') (N : ℕ -> nGate' m' o)
  : nfold_tensor_product n M * nfold_tensor_product n N
  = nfold_tensor_product (n) (λ i => M i * N i) := by
  induction n with
  | zero => simp [nfold_tensor_product]
  | succ m ih =>
    rw [nfold_tensor_product, nfold_tensor_product, nfold_tensor_product]
    rw [← reindex_3_mul]
    rw [← mul_kronecker_mul]
    rw [ih]
```

As can be seen, we need to handle the reindexing explicitly, which is something that will shortly be discussed in Subsection 4.2.3 and in more detail in the Section 4.5. We apply the n -fold tensor product definition and then we apply the `mul_kronecker_mul` definition that shows that $(A * B) \otimes (C * D) = (A \otimes C) * (B \otimes D)$.

The other three theorems are simpler but still useful. We prove that the n -fold tensor power of an identity gate becomes a larger identity gate in `nfold_tensor_product_of_one`. We prove that the n -fold tensor product (power n) of a `natCast` of a natural number i as a multiple of an identity gate is equivalent to the identity gate of the resulting size multiplied by i^n . And finally, we prove that we can extract a scalar multiplier c from an n -fold tensor product by multiplying it with c^n . These all also have a variant for the `tensor_power` shorthand.

4.2.3 Reindex

The reindexing of matrices is required to tell Lean that matrix dimensions are identical, and to rewrite them in a neat way. In a perfect world, Lean would not have trouble managing the dimensions of matrices and would infer their equivalence. More on this will be discussed in the Difficulties section, Section 4.5.

For this thesis, it has been decided that sticking with the working but impractical solution best fit in the timeline and scope. I will cover the different equivalences between ways of defining the dimensions of matrices first. Then, I will cover the different theorems on `reindex` that can be found in Quantum Lean.

```

def QCount_mul_QCount : (QCount m × QCount n) ≈ QCount (m + n)

def QCount_mul_one : (QCount (m * 1)) ≈ QCount m

def QCount_mul_succ_n : (QCount (m * n) × QCount m) ≈ QCount (m * (n + 1))

def QCount_assoc : (QCount (m * n * o)) ≈ QCount (m * (n * o))

def QCount_mul_add : (QCount (m * n) × QCount (m * o)) ≈ QCount (m * (n + o))

```

Above are the five equivalences for matrix dimensions, without their respective proofs for brevity. Let us also recall that $\text{QCount } n = 2^n$.

The first equivalence shows the obvious relation that multiplying QCounts is equivalent to adding the number of qubits. The second shows that multiplying the number of qubits by one does not affect anything. Its reason for existing is that Lean does not automatically handle the normalisation of types.

The third equivalence is a little more advanced. It is used in `reindex3` and thus in the `nfold_tensor_product` definition. It shows that applying a tensor power to a matrix with a size of $m \cdot n$ and a matrix of size m , its resulting size is equivalent to $m \cdot (n + 1)$.

Finally, the fourth equivalence is another normalisation example, as Lean does not handle the normalisation of types. The last equivalence is meant for use in a suggested new theorem on tensor powers, discussed in Chapter 5, Section 5.1.

For each of these equivalences, we define a `reindex`, indexed with a number in subscript (e.g. `reindex1`). Then, we define a set of four theorems for each `reindex`.

- `reindex_one`
- `reindex_mul`
- `smul_reindex`
- `reindex_natCast`

The first theorem shows that whenever we have a `reindex 1`, it is equivalent to 1. The second theorem shows `reindex (A * B) = reindex A * reindex B`. This second theorem is very useful to clear reindexes out of the way to apply other theorems.

The third theorem shows that any scalar multiplication may happen inside or outside of the `reindex`. The final theorem shows that for any `natCast i` or natural number as diagonal matrix, it is equivalent to this same `i` without reindexing.

4.3 Gates

With all the building blocks now explained, we continue to the gates defined in Quantum Lean. To avoid repetition, I will consider the Hadamard gate in detail, note any differences for the Pauli gates, and finally write a few more things on conditional gates.

4.3.1 Hadamard

In Quantum Lean, gates are defined simply by their matrix representation. For Hadamard gates, that looks like so.

```

def H : nGate 1 := !![1, 1; 1, -1]

```

It is a 2×2 matrix as indicated by the custom type `nGate 1`. Due to limitations, gates are not normalised. More on this in Section 4.5.

Based on the definition of H , here is a definition $H^{\otimes n}$.

```
def H_n (n : ℕ) := tensor_power n H
```

This definition utilises the tensor power definition.

Then, there is the H_i definition.

```
def H_i (n i : ℕ) := apply_at n i H
```

This definition allows the application of a Hadamard gate on a single qubit at index i .

We then define two theorems, the identity theorem $H \cdot H = 2$ (where the 2 is a `natCast` to a 2×2 diagonal matrix).

```
theorem H_Identity : H * H = (2 : ℕ) := by
  norm_num [H]
  simp only [ofNat_fin_two]
```

Then, we define a recursive proof for this statement.

```
theorem H_mul_H : H_n n * H_n n = (2 ^ n : ℕ) := by
  rw [H_n, nfold_tensor_product_mul, H_Identity, nfold_tensor_product_of_natCast]
```

In general, theorems surrounding important properties that are required or useful for proofs should be supplied adjacent to the definition of the gates they concern.

4.3.2 Pauli

For each of the Pauli gates, we provide identical definitions and theorems as Hadamard. As with the Hadamard gate, the square of a Pauli gate is equal to the identity gate, but this time, we can define the gates normalised, so we do not need to equate it to $2 \cdot I$.

For Pauli gates, we can define two more circuit equivalences; $H \cdot X \cdot H = Z$ and $H \cdot Z \cdot H = X$. As their proofs are similar, here are the proofs for the single-gate and tensor power versions for the first equivalence.

```
theorem HXHegZ' : H * X * H = 2 * Z := by
  rw [H, X, Z]
  norm_num
```

```
theorem HXHegZ : H_n n * X_n n * H_n n = (2 ^ n : ℕ) * Z_n n := by
  rw [H_n, X_n, nfold_tensor_product_mul, nfold_tensor_product_mul]
  -- nfold_tensor_product n (((fun x => H) * fun x => X) * fun x => H) = 2 ^ n * Z_n n
  rw [HXHegZ', nfold_tensor_product_smul, Z_n]
```

As is visible, the upper proof for the single gates is done with `norm_num` and thus calculated. This is fine on this scale. For the arbitrarily sized or tensor power case, we prove it by applying a handful of the previously discussed theorems.

First we rewrite the definitions for $H^{\otimes n}$ and $X^{\otimes n}$. We apply `nfold_tensor_product_mul` twice to turn three n -fold tensor products into one. We can rewrite in the first theorem we defined, then pull out the scalar multiplication using the `nfold_tensor_product_smul` theorem, and prove its equality by substituting in $Z^{\otimes n}$.

4.3.3 Conditional

Finally, we have the conditional gates that work on multiple gates at a time. Currently, there are implementations for the controlled X and Z gates. These gates also have their respective $CX^{\otimes n}$ and $CZ^{\otimes n}$ defined, as well as theorems for their identity when applying the gate twice.

Finally, since a basic SWAP gate is made solely from CX and reversed CX gates (denoted XC), this gate has also been defined.

```
def SWAP : nGate 2 := CX * XC * CX
```

```
theorem SWAP_mul_SWAP : SWAP * SWAP = (1 : ℕ) := by
  simp [SWAP, CX, XC]
  ext i j
  fin_cases i <> fin_cases j <> rfl
```

4.4 Formalising Algorithms

With the gates defined, it is possible to start formalising quantum algorithms. For each algorithm, I will indicate what each of the definitions for the algorithm do and how they fit together into the algorithm *theorem* at the end.

4.4.1 Deutsch

As previously defined, the circuit representation for the Deutsch algorithm can be found in Figure 4.3. The oracle gate O_s for accessing the bits of the string $s = s_0s_1$ is defined as follows:

$$O_s = \begin{bmatrix} (-1)^{s_0} & 0 \\ 0 & (-1)^{s_1} \end{bmatrix}.$$

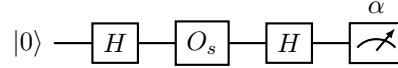


Figure 4.3: The Deutsch algorithm.

The output we expect should be $|s_0 \oplus s_1\rangle$, so $|0\rangle$ if $s_0 = s_1$, and $|1\rangle$ if $s_0 \neq s_1$.

First of all, we define the `Oracle` in Lean. Instead of using s as a bitstring, I have opted for two bits a and b , which represent the individual bits of s , that is $s = ab$, or $a = s_0$ and $b = s_1$.

```
def Oracle (a b : Bool) : nGate 1 := !![(-1) ^ a.toNat, 0; 0, (-1) ^ b.toNat]
```

Next, we define the final state of the qubit which we denote by `Outcome`. This will be equal to $|0\rangle$ if $a = b$ and $|1\rangle$ if $a \neq b$. To calculate this, I used the same powers of -1 as before, since their addition or subtraction would net this same result, although multiplied by two. Since we need to accommodate a factor of two due to the non-normalised Hadamard gates in the algorithm, the division and multiplication cancel out automatically.

```
def Outcome (a b : Bool) : nQubit 1 :=
  !![(-1) ^ a.toNat + (-1) ^ b.toNat, (-1) ^ a.toNat - (-1) ^ b.toNat]
```

Finally, we define the algorithm itself.

```
def ZeroQubit : nQubit 1 := !![1, 0]

theorem DeutschAlgorithm (a b : Bool) :
  ZeroQubit * H * (Oracle a b) * H = Outcome a b := by
  rw [ZeroQubit, H, Oracle, Outcome]
  fin_cases a <|> fin_cases b <|> norm_num
```

As can be seen, the Lean code for the oracle neatly mimics the circuit drawn in Figure 4.3. This is done intentionally to reduce the chance at mistakes when creating formalisations of algorithms.

We substitute `ZeroQubit`, `H`, `Oracle` and `Outcome` for their definitions, and then go through each case of a and b to prove that this equality holds.

4.4.2 Bernstein–Vazirani

The Bernstein–Vazirani algorithm, with its circuit represented in Figure 4.4, is the second algorithm proven correct in Quantum Lean. Unlike Deutsch, it is not possible to go through each possibility, as this algorithm performs actions on an arbitrary number n of qubits.

The oracle in this case is the n -fold tensor product $Z^{s_0} \otimes \dots \otimes Z^{s_{n-1}}$. To transform this into Lean code, I first define `Oracle` function, which returns the Z^{s_i} gate given the input bit s_i . Then, I define `Oracles` which receives s as a function that for a given natural number j outputs the corresponding boolean value s_j , which makes the reading of the bitstring much easier. The `Oracles` returns a function that for a given input i returns the gate for the corresponding qubit.

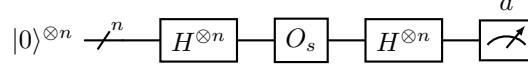


Figure 4.4: The Bernstein–Vazirani algorithm circuit.

```
def Oracle (si : Bool) : nGate 1 := Z ^ si.toNat
def Oracles (s : ℕ → Bool) : (i : ℕ) → nGate 1 := fun i => Oracle (s i)
```

Then, we define the desired final state $\bigotimes_{i=1}^n X^{s_i} |0\rangle$ which we call Outcome. Given the same function s , it returns a function with input i and output $X^{s_i} |0\rangle$. If $s_i = 0$, the function returns $|0\rangle$, and if $s_i = 1$, it returns $|1\rangle$. It does this by multiplying the $|0\rangle$ or QZero with X^{s_i} , which is equal to I if $s_i = 0$ and X if $s_i = 1$.

```
def Outcome (s : ℕ → Bool) : (i : ℕ) → nQubit 1 :=
  fun i => 2 • QZero * X ^ (s i).toNat
```

Then, we create two helper theorems. These handle the cases where either $s_i = 0$ or $s_i = 1$. For the first case, it boils down to Z^0 and X^0 becoming the identity gate, and the H matrices cancelling out. The latter case applies the $H \cdot Z \cdot H = X$ identity.

```
theorem si_eq_zero : H * Z ^ Bool.toNat false * H = 2 • X ^ Bool.toNat false := by
  rw [Bool.toNat_false, pow_zero, mul_one, H_Identity, pow_zero, @nsmul_one]
```

```
theorem si_eq_one : (H * Z ^ Bool.toNat true * H) = 2 • X ^ Bool.toNat true := by
  rw [Bool.toNat_true]
  simp only [pow_one]
  exact HZHegX'
```

Finally, we have the algorithm itself, again, mimicking how the circuit looks. It receives the s function that will be passed to both Oracle_s and Outcome . We apply the QZero_n and H_n definitions. Then, we rewrite the multiplication of tensor powers in terms of the multiplication of the individual gates put into a single tensor power. We continue with an induction, proving the base case by showing that with $n = 0$, there is no gates so the equality holds. For the successive case, we apply the definition of the n -fold tensor product.

```
theorem BernsteinVaziraniAlgorithm (s : ℕ → Bool) :
  QZeron * (Hn * (nfold_tensor_product n (Oracles s)) * Hn)
  = nfold_tensor_product n (Outcome s) := by
  rw [QZeron, Hn]
  rw [nfold_tensor_product_mul, nfold_tensor_product_mul, nfold_tensor_product_mul]
  induction n with
  | zero => rw [Nat.zero_eq, nfold_tensor_product, nfold_tensor_product]
  | succ n ih =>
    -- ih : (nfold_tensor_product n fun i => QZero * (H * Oracles s i * H))
    --       = nfold_tensor_product n (Outcome s)
    rw [nfold_tensor_product, nfold_tensor_product]
    -- (nfold_tensor_product n fun i => QZero * (H * Oracles s i * H)) ⊗k (Oracles s i * H)
    -- = (nfold_tensor_product n (Outcome s)) ⊗k (Outcome s (n + 1))

    rw [ih, Oracles, Oracle, Outcome]

    cases (s (n + 1));
    rw [si_eq_zero]
    rw [si_eq_one]
```

By applying the definition of the n -fold product, we reduce to the n case (as we are now in the $n + 1$ case) and this allows us to apply the inductive hypothesis ih now. After rewriting three more definitions, we go through the two cases for s_{n+1} . For each of these cases, we defined a helper theorem, so we apply each of these and close the goal. The algorithm is proven correct for an arbitrary number of qubits.

4.4.3 One-out-of-Four Grover

Finally, the last algorithm we formalise is the one-out-of-four Grover search. This algorithm is richer in its use of gates, as can be seen in Figure 4.5 where the circuit for the algorithm is once again displayed.

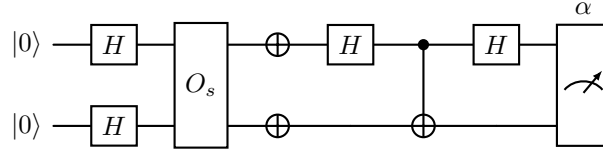


Figure 4.5: The one-out-of-four Grover search algorithm circuit.

To handle the secret input string s , I created a new type abbreviation `sType` that defines a number from 0 to 3. Then I can treat the string s as a function $s : \{0, 1, 2, 3\} \rightarrow \{0, 1\}$. With this, we can define the oracle.

```
-- s is a number from 0 to 3 -/
abbrev sType := Fin 4

def Oracle (s n : sType) : C := fun n => (-1) ^ ((s == n).toNat)
def Oracle_s (s : sType) : nGate 2 :=
  !![Oracle s 0,      0,      0,      0;
     0, Oracle s 1,      0,      0;
     0,      0, Oracle s 2,      0;
     0,      0,      0, Oracle s 3]
```

Then, the desired final state `Outcome` is defined, with a scalar multiple of 4 to accommodate the non-normalised Hadamard gates.

```
def Outcome_i (s n : sType) : C := 4 * (s == n).toNat
def Outcome (s : sType) : nQubit 2 :=
  !![Outcome_i s 0, Outcome_i s 1, Outcome_i s 2, Outcome_i s 3]
```

Finally, we define the algorithm itself. Note that I created short-hands for the following 2-qubit gates: $HH = H \otimes H$, $XX = X \otimes X$ and $HI = H \otimes I$. We again start by rewriting the definitions of our gates. We utilise `norm_num` to simplify the matrix multiplication and apply the `Oracle_s` definition. Then, we define four goals using `fin_cases s` for each possible value of s . We need to use `simp` with a host of theorems to make Lean replace the finite case objects with their values. Finally, we use `norm_num` to indicate that the matrices on both sides of the equation are equal.

```
theorem OneOutOfFourGroverAlgorithm (s : sType) :
  QZero_2 * HH * Oracle_s s * XX * HI * CX * HI = Outcome s := by
  rw [QZero_2, HH, Oracle_s, XX, HI, CX, Outcome]
  norm_num [Oracle]
  fin_cases s <|> {
    -- Turn the finite case objects into its natural number values
    simp only [Fin.reduceFinMk, Fin.isValue, Fin.reduceBEq, Bool.toNat_false, pow_zero,
      beq_self_eq_true, Bool.toNat_true, pow_one, add_left_neg, add_right_neg,
      add_zero, neg_neg, Outcome_i]
    norm_num
  }
```

4.5 Difficulties

This thesis was not without its hurdles. Throughout the process of formalising algorithms in Lean, I walked into numerous problems and had my fair share of setbacks. Some had relatively easy workarounds or solutions, and some did not. I would like to quickly go over the main difficulties I faced, to indicate my reasons for certain decisions. For some of these, I will also link to the Further Work chapter, Chapter 5, as I believe there are solutions that may in the end be more fruitful but were outside of the scope of this thesis.

One of the first obstacles I ran into was the normalisation of gates. As indicated, the Hadamard gate is not normalised. This is because I could not find a suitable way of handling a scalar $\frac{1}{\sqrt{2}}$ multiplication, as the usage of roots in Lean require definitions and theorems to be marked `non-computable`, which disables a lot of the functionalities I am relying on. For example, the basic property of $H \cdot H = I$ was not possible to prove. This would have meant I could not progress. The way I continued was accepting that scalar multiplications based on gate usage was a necessary evil. This is why in some places you can find extra factors of 2 or even 2^n .

The next difficulty I faced was the reindexing of matrices. Initially, I used to have a single definition for `reindex` and it would get in the way of many proofs I was writing. This amount of reindexes ended up growing to four definitions. I first thought the amount of reindex definitions that needed to be made would grow even larger than this, and it looked like I was going to need to prove theorems between each reindex, creating an exponential amount of code.

However, the first one fell into disuse after further generalisation of the equivalences behind these reindexes. Because of this generalisation, I found that the numerous theorems that required these reindexes also ended up hiding the requirement of the reindexes. While I still needed to keep these reindexes in mind proving theorems, the definitions and theorems on reindexes were clear and easy to apply. In the formalisation of algorithms, these reindexes do not show up often anymore, usually only when rewriting theorems that apply a reindex, such as `tensor_power`.

The appearance of `reindex` at some times is still impractical, as it distracts from the greater goal to be solved. It makes Lean goals harder to read, and harder to prove, as we need to handle reindexing. In a perfect world, we can design the `Matrix` in `mathlib` in such a way that Lean can automatically reshape tensors of smaller gates into an appropriately-sized gate. A possible solution for this is discussed in Chapter 5, Section 5.4.

Finally, a more minor issue that I faced was that Lean did not support using `#eval` with complex numbers. I ended up submitting a merge request on the `mathlib` GitHub repository adding the necessary code after writing it for Quantum Lean. The evaluation is not as clean as I would have liked it, but due to `mathlib`'s way of defining real numbers as Cauchy sequences, I was only able to represent a complex number as follows.

```
def two : ℂ := 2
#eval two
-- Real.ofCauchy (sorry /- 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, ... -/) +
-- Real.ofCauchy (sorry /- 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ... -/)*I
```

In the future, the representation definitions found in `mathlib` can perhaps be overwritten to create a near-perfect but flawed representation that is as compact as $p + qi$. The details of this will be discussed in Chapter 5 as well, in Section 5.3.

4.6 Documentation

The most important part of a project such as this is ample documentation. This gives one the ability to contribute without first having to familiarise themselves for many hours by reading through a code-base they are not familiar with.

While Chapters 2 and 3 provide a good starting point for those unfamiliar with either the theory or Lean, I believe written documentation next to the code is merited. This will make it easier to follow along, as the documentation does not have to be looked up from outside of the code-base. The documentation can also be fine-tuned and condensed, and updated over time.

So what documentation does Quantum Lean provide?

- Information on understanding Lean, including links to learning material and useful tools to use during development;
- A list of common `mathlib` files that are used across Quantum Lean;
- Explanations on file structure and theorem layout;
- General explanations in each file explaining the functionality of its content;
- Explanation of the formalised algorithms alongside the code.

As can be assumed from the fourth point, Quantum Lean will not add comments to every single theorem that it has. This would cause a lot of duplication in explanation, and the possibility for the documentation to become outdated is higher. The general explanation also contains more higher-level information that can get lost when describing singular theorems or definitions.

Further Work

Quantum Lean is a foundation that I have set up from the bottom up. Therefore, I have determined venues to explore for how to improve the foundation and in which direction development could be steered to next. I will describe a set of improvements that can be made to further refine Quantum Lean and make it more powerful and capable to formalise increasingly complex algorithms.

To avoid the obvious, I will not consider any new algorithms to formalise. As a general rule, algorithms should be tackled in increasing complexity, perhaps in the same order as they were discovered. This will naturally build up a greater list of theorems that can be used across algorithms, as simpler equalities such as $H \cdot X \cdot H = Z$ and $H \cdot Z \cdot H = X$ are extracted into their own theorems.

What this section *will* focus on is the foundational improvements that will allow for easier proof-writing. For example, new definitions surrounding the tensor power, as discussed in Section 5.1, should allow for better reordering of tensor powers so more advanced multi-qubit circuits can be transformed. In Section 5.2, I will discuss an important notation shorthand that can be implemented to allow for easier definition of qubit states. In Section 5.3, I will discuss the possibilities in making `#eval` work more efficiently for the definitions in Quantum Lean and how it can be made more legible.

The final two sections, Section 5.4 and 5.5, are more theoretical and more complex undertakings. The first of these sections will be on the redefinition of the `Matrix` type from `mathlib` to perhaps make it easier to utilise. The latter section is a quick word on the possible implementation of the quantum Hoare logic, and its type system.

5.1 Theorems on Tensor Power

While Quantum Lean already contains a set of useful and general theorems on the tensor power, certain operations will require more advanced ways of handling tensor powers that I have not yet had the need to use. To pave the way for more of these theorems to be written, I did consider the potential needs for new tensor power theorems. While I believe the need for more advanced theorems will come naturally alongside the formalisation of algorithms, expanding past the required is never a bad idea. This idea drove the generalisations I have made in my theorems so they can apply in many cases.

With the suggested code in this section, I will provide theorems that are written in a general way. Their exact form may perhaps change slightly based on the implementation, but it should be done with caution, to avoid steering away from the generalisation they already contain. The theorems will also contain reindexes. Depending on the work done based on Section 5.4, this will need to be changed.

The first theorem is `nfold_tensor_product_one`, which is to show that the following equivalence holds: `nfold_tensor_product 1 gates = gates 1` itself. With this theorem, one can extract singular gates and apply theorems on them.

```
-- nfold_tensor_product 1 of any gate is equivalent to that gate
theorem nfold_tensor_product_one {m m' : ℕ} { gates : ℕ -> nGate' m m' } :
  reindex₂ (nfold_tensor_product 1 gates) = gates 1
```

With this theorem, one can consider even more theorems or functions. For example, being able to prove that a tensor power is equivalent to some matrix can be useful at times—as long as this is done within reasonable number of qubits, i.e. for gates that apply to a couple of qubits maximum. For these sorts of theorems, or helper functions, I have not considered exactly how to write them in Lean, as this is very dependent on how `nfold_tensor_product` interacts with `Matrix`, and how Lean functions in general.

Next up is a set of theorems to reorder a tensor power. The current tensor power is defined left-to-right in the equation. This means that a new gate `gates (n + 1)` is added to the right of `nfold_tensor_product n gates`. This makes it easy to extract the final gate from a tensor power, but hard to do the same with the first gate. Therefore I suggest the following theorem.

```
-- Slice a tensor power in two pieces
theorem nfold_tensor_product_slice (gates : ℕ -> nGate' m m') :
  nfold_tensor_product (n + o) gates =
    reindex₅ (nfold_tensor_product n gates ⊗ₖ
      nfold_tensor_product o (fun i => gates (n + i)))
```

This theorem allows one to metaphorically cut a tensor power in two, or perhaps glue two tensor powers together. With this general theorem, it is possible to separate the first gate from the rest, or use it twice to cut out a specific section in the middle of a tensor power. When utilising multi-qubit gates, I see this theorem being used quite a bit. Note that the `reindex?` is a reindex that is not defined yet and needs to be considered in the proof of this theorem.

Perhaps in extension of the previous theorem is `tensor_power_of_tensor_power`, which is focussed more on `tensor_power`. This theorem proves that the kronecker product of two tensor powers is equivalent to having one single tensor power, when using identical gates.

```
-- Turn a single tensor_power into a tensor_power of a tensor_power
theorem tensor_power_of_tensor_power (M : nGate' m m') :
  tensor_power (n * o) M = reindex₄ (tensor_power o (tensor_power n M))
```

This theorem states that if one has a `tensor_power` of some gate M , we can split it up as follows: $M^{\otimes n*o} = M^{\otimes n} \otimes M^{\otimes o}$. Together with the `tensor_power_slice` theorem, this can be used to define certain circuit sections effectively and efficiently.

Finally, I'd like to suggest either creating a more general version of `apply_at`, which creates a tensor power that only contains identity matrices except for a given index, on which it contains gate M .

The current definition works well for single-qubit gates, but falls short for multi-qubit gates, as the identity gate used is identically sized. Currently, it is not possible to define a tensor power with an n -qubit gate that applies on any set of qubits of which the first qubit is *not* a multiple of n .

Creating a definition that does this is hard, because it requires mixing differently sized gates: 1-qubit identity gates and an arbitrary n -qubit gate. This may require further generalising the `nfold_tensor_product` and `tensor_power` definitions. As I have come to learn, a lot of experimentation usually brings new ideas and solutions, which I suggest for this problem as well.

All in all, there are enough theorems that still need to be specified and proved. I have taken the liberty to add the mentioned code in the above code-blocks into the source code, using the `sorry` command in Lean to indicate that the theorems are not yet proven.

As for the code, this can be found under `Data/Circuits/Tensor.lean` in Quantum Lean. This file contains the definition for `nfold_tensor_product`, `tensor_power` and all of the theorems that have been written for it.

5.2 Dirac Notation

Another improvement that can be made to Quantum Lean is support for the Dirac notation, or *Bra-Ket* notation. This notation will make it easier to work with different types of qubit states.

Defining a tensor power of n zero qubit states should be doable with as little hassle as possible. Just like the tensor product of n qubits, or an arbitrary string of 0s and 1s. Even the $|+\rangle$ state can be supported, and perhaps even shorthands for more complex symbols such as Bell states, too.

In any event, the Dirac notation needs to be made in Lean using the meta-programming capabilities of Lean. This allows the creation of custom syntax that we need to define the $\langle x|$ (*bra*) and $|x\rangle$ (*ket*) symbols and the way these notations function. This will likely be done using macros.

The Dirac notation is related to the `Matrix` type, because a qubit in Quantum Lean is defined as a matrix, as well as to binary numbers, which are put inside the Bra-Kets at times. Therefore, the `Mathlib/Data/Matrix/Notation.lean` and `Mathlib/Data/Nat/Bits.lean` files will serve as a good first starting point for the implementation of the Dirac notation. The former file contains the `!![]` notation that is able to define a matrix. The latter file contains a `mathlib` implementation for bits, which may be useful for the parsing of bits inside of the Bra-Kets.

Furthermore, I'd suggest going through the `mathlib` codebase to find more `Notation.lean` files. These contain custom `mathlib` specific notations for a host of different areas of mathematics. These may contain even more useful macros that do something similar.

5.3 #eval

This improvement to Quantum Lean is mostly a quality of life improvement. The current definitions in `mathlib` that Lean applies when evaluating `Matrix` and `ℂ` types are very verbose. It also requires us to reindex our dimensions when attempting to evaluate a Kronecker product. These changes may make it easier to work with `#eval`.

It is possible to define a `Repr` or representation definition for each type, that defines how it should get formatted when evaluated using `#eval`. For matrices, this representation lives in the same `Mathlib/Data/Matrix/Notation.lean` file. This definition is written with the following matrix type:

```
... : Repr (Matrix (Fin m) (Fin n) α) where ...
```

This snippet of code defines a representation for any matrix defined with a single `Fin` instance for each dimension. This explains why Lean has no problem representing any `nGate`, but the moment a Kronecker product comes along, we first need to reindex. For example:

```
def H : nGate 1 := !![1, 1; 1, -1]
def HH := H ⊗k H

#check HH -- HH : Matrix (QCount 1 × QCount 1) (QCount 1 × QCount 1) ℂ
#eval H ⊗k H
-- Instance error: type has no implementation for Repr
-- (Error summarised for brevity)
```

Since we have the required code to deal with reindexing without much issue, I don't want to suggest a more advanced representation that can utilise any dimension-type matrix. Instead, I would like to suggest more pressing issues: verbosity. If we evaluate the `H` up top, we do not just get `!![1, 1; 1, -1]`:

```
#eval H
-- !![Real.ofCauchy (sorry /- 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ... -/) +
--   Real.ofCauchy (sorry /- 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ... -/)*I,
--   Real.ofCauchy (sorry /- 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ... -/) +
--   Real.ofCauchy (sorry /- 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ... -/)*I;
--   Real.ofCauchy (sorry /- 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ... -/) +
--   Real.ofCauchy (sorry /- 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ... -/)*I,
--   Real.ofCauchy (sorry /- -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, ... -/) +
--   Real.ofCauchy (sorry /- 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ... -/)*I]
```

We already get a wall of Cauchy sequences solely because our `Matrix` definition utilises the `C` type for its values. Imagine what a 2-qubit or 3-qubit matrix would look like. Recall that a single complex number is represented using two Cauchy sequences for each of its $p + qi$ values:

```
#eval (1 : C)
-- Real.ofCauchy (sorry /- 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ... -/) +
-- Real.ofCauchy (sorry /- 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ... -/)*I
```

I contributed this complex number representation and had it merged into `mathlib` so there would be *any* representation of a complex number as opposed to none. However, I feel that its verbosity combined with the amount of complex numbers found in a matrix makes it hard to use at best, for Quantum Lean’s use-case. While the maintainers of `mathlib` rightfully do not want to trade the accuracy for an approximated output, I do believe there is a way to make this more readable in Quantum Lean itself.

It should be plausible to override the `mathlib` definition of real numbers in Quantum Lean to replace the Cauchy representation with something easier to read. This would start with understanding that—in a very simplified way—a *real* number is just a Cauchy sequence of *rational* numbers approaching the value of said real number.

Diving through the definitions, it should be possible to take, say, the 100th number in the Cauchy sequence, which is a rational number, and print only that number. A recent merge of [PR #750](#) from the `batteries` library for Lean implements an exact conversion between rational numbers and floats. This is important because of the following representation:

```
def rational : Rat := 3.5
#eval rational -- (7 : Rat)/2
```

A rational number is defined as a division. While this again provides accuracy, it adds a lot of bulk to any representation of a rational number. With this new function from the `batteries` library, it is possible to create a single readable number.

```
#eval rational.toFloat -- 3.500000
```

If this is used to represent the rational number from the Cauchy sequence, and this new definition of the Cauchy sequence utilised for the complex number sequence, our evaluation will be much shorter:

```
#eval (1 : C) -- 1.000000 + 0.000000*I

#eval H --
-- !![1.000000 + 0.000000*I, 1.000000 + 0.000000*I;
-- 1.000000 + 0.000000*I, -1.000000 + 0.000000*I]
```

Finally, if rounding to two or three decimals is supported and implemented, the representation would shrink even more. The complex number representation could also hide the real or imaginary part of the number if the rational number equals zero, making the evaluation as concise as possible, while perhaps only losing marginal accuracy at best.

5.4 Redefining Matrices

There is one change that could theoretically make Quantum Lean even easier to use: redefining the `Matrix` type. Based on conversations with the Lean community in this [Zulip conversation](#), it seems that the implementation of functors for the dimensions of matrices *may* make reindexing obsolete, due to the functor dimension being a non-strict symmetric monoidal category. While this would be great and while it would clean up parts of the codebase and make proofs a bit easier, it is not a fact that it will provide the functionality we are looking for. This needs to be properly and thoroughly researched.

However, based on my initial parsing and understanding of the code in the code-base of CoqQ (L. Zhou et al. 2023), I believe the same theory *is* used in CoqQ for their definition of matrices. This would indicate that there *may* be merit in the pursuit of this implementation.

However, as previously mentioned, the current impact of the use of reindexes is limited, and mostly hidden within other theorems that Quantum Lean defines. It should be heavily considered

whether the effort of handling reindexes is larger or smaller than the effort of reimplementing matrices with functor dimensions.

Regardless, I have considered the option and checked its viability by seeing whether I could at least port parts of the `Mathlib/Data/Matrix/Basic.lean` definitions and theorems to implement a possible version of functors for dimensions of the `Matrix` definition. I have added this file into the `functor-matrix` branch of Quantum Lean to not clog the final product.

Further considerations and implementation should be done after thorough research and perhaps preliminary testing to see if this implementation would actually grant us all we need from it before porting the entire set of `Matrix` definitions and theorems, as well as `Kronecker` definitions and theorems.

5.5 Quantum Logic

Finally, I'd like to shortly mention the quantum Hoare logic, as well as the type theory for Hoare logic. I believe moving towards an implementation of QHL will provide an even more rigorous foundation for algorithms to be formalised in.

I believe this implementation will require a large amount of time and resources to be implemented, based on the paper and code-base for CoqQ, the Coq implementation of quantum Hoare logic (L. Zhou et al. 2023). Add to this the type theory put forward in Singhal 2020, which will require even more work to implement.

I believe Lean does provide a stable and rich foundation for implementing this logic and its type theory. Perhaps the type class system built into Lean may be of use. These define certain extra constraints for types, such as the decidable equality, as seen in Subsection 4.2.1.

Conclusions

This thesis focussed on creating a foundation in which quantum algorithms can be formalised. It utilised the Lean theorem prover and programming language with its mathematical library `mathlib` to apply the rich set of existing theorems and proofs to the new task of formalising quantum algorithms. For Quantum Lean, a new set of more advanced and specific theorems and definitions was created. These have been applied in theorems to formalise a set of three algorithms. With these formalisations, I have demonstrated that Quantum Lean is capable of both arbitrarily-sized algorithms as well as algorithms with controlled gates.

As has been observed in the formalisation of the current three algorithms, the formalisation of equalities such as $H \cdot X \cdot H = Z$ allows us to build up a collection of theorems to apply in larger and more complex circuits and algorithms. With Lean allowing parallel checking of proofs, the Quantum Lean library should retain a swift execution time, while providing a rich selection of theorems to apply to a growing collection of algorithms.

With the intuitive functions written specifically for Quantum Lean, it allows for an easier time formalising algorithms while hiding a large amount of mathematical theorems proven in `mathlib`. This is imperative for adoption.

Equally so is the documentation of a product. Quantum Lean contains a rich set of documents explaining its workings with links straight to source-code, so anybody can begin to use Quantum Lean easily. There are references for learning Lean, as well as explanations of files and the quantum algorithms currently proven.

All in all, Quantum Lean provides a solid foundation for the proving of correctness of quantum algorithms. And with the detailed documentation of further work in Chapter 5, I believe Quantum Lean can become even more powerful.

Acknowledgements

There are quite a number of people that I would like to express my gratitude to, as each of the names I will mention helped me on my path, either to understanding Lean, applying it more efficiently, solving problems I faced and more. In no particular order, I thank these people:

- Riccardo Brasca, for pointing me to `norm_num` and teaching me how to make `simp` more powerful;
- Yaël Dillies, for improving my matrix understanding;
- Eric Wieser, for telling me about Loogle, creating PRs for missing theorems, helping me polish my own PR, and giving me hints;
- Junyan Xu, for pointing me to `Matrix.reindex`;
- Mitchell Lee, for helping me prove the $H \cdot H = I$ identity for n qubits;
- Kyle Miller, for pointing me to `prod_assoc%`, and discussing the alternative matrix definition using functors with me.

Finally, I most of all want to thank Māris Ozols, my supervisor for this thesis. We met up weekly to discuss my progress, where he consistently helped me navigate my formalisation journey. He indicated what I could tackle next, writing down many formulae on his whiteboard, going through the theory so we were both on the same page. He endured the hours of my explaining Lean and its different functions, and what my code does and why. Finally, he combed through my entire writing to make a plethora of remarks, suggestions and inquiries, all to improve and elevate my writing. Thank you, Māris.

7.1 AI

I'd also like to acknowledge a growing ethical concern within the world, namely AI. On this topic of AI, I can be concise. There has been no use of Large Language Models such as ChatGPT in the writing of this thesis. Furthermore, while I have had GitHub Copilot installed in VS Code, I found none of its suggestions of use and disabled it early on in development. Thus, my code is also free of AI-generated code.

Now, I would like to be more elaborate. While I believe AI can be a great tool for certain tasks, I often see the limitations in its use. Copilot is obviously trained on the giant catalogue of Lean 3 code, so its suggestions in Lean 4 are sorely lacking. ChatGPT can write enormous lengths of text on any subject, but its validity is dubious, and its length quickly becomes wordy or repetitive. This does not even tackle the biggest ethical critiques on Large Language Models: infringement of copyright, and in extension plagiarism, the disinformation that is (confidently) presents and the environmental footprint that its training has on nature.

- **Infringement of copyright:** LLMs do not provide the sources used for its output, which may sometimes be a single source which is copied verbatim. This can fall squarely into plagiarism territory, a place best avoided in academic writing.
- **Disinformation:** LLMs have a tendency to *hallucinate*, meaning it presents false or made-up information as factual and true, at times even confidently. For people unaware of this fact, this can have a profound negative impact, as they start believing these factual inaccuracies or lies that the LLMs output.
- **Environmental footprint:** the training of an LLM requires extensive amounts of time and power, which means it has an incredibly large environmental footprint for the work it provides. I find the benefits of LLMs thoroughly lacking when compared to the footprint it has.

All in all, I do not use Large Language Models, as I believe the drawbacks outweigh the benefits by magnitudes. I do not use it for academic writing, and only use it once in a blue moon in daily life.

Bibliography

- Barbosa, Manuel et al. (2023). *Fixing and Mechanizing the Security Proof of Fiat-Shamir with Aborts and Dilithium*. Cryptology ePrint Archive, Paper 2023/246. URL: <https://eprint.iacr.org/2023/246>. URL: <https://eprint.iacr.org/2023/246>.
- Barthe, Gilles et al. (2011). “Computer-Aided Security Proofs for the Working Cryptographer”. In: *Annual International Cryptology Conference*. URL: <https://api.semanticscholar.org/CorpusID:924868>.
- Bloom, Thomas F. (2021). “On a density conjecture about unit fractions”. In: URL: <https://api.semanticscholar.org/CorpusID:244920547>.
- Breitner, Joachim (Aug. 2023). URL: <https://loogle.lean-lang.org/>.
- Dillies, Yaël (Mar. 2023). *LeanAPAP*. URL: <https://yaeldillies.github.io/LeanAPAP/>.
- EPSRC (July 2023). *Details of Grant*. URL: <https://gow.epsrc.ukri.org/NGBOViewGrant.aspx?GrantRef=EP%2FY022904%2F1>.
- Gowers, William T. et al. (2023). “On a conjecture of Marton”. In: URL: <https://api.semanticscholar.org/CorpusID:265128704>.
- Hartnett, Kevin (Sept. 2016). *Hacker-proof code confirmed*. URL: <https://www.quantamagazine.org/formal-verification-creates-hacker-proof-code-20160920/>.
- Jordan, Stephen (Apr. 2011). URL: <https://quantumalgorithmzoo.org/>.
- Kubota, Takahiro et al. (2016). “Semi-automated verification of security proofs of quantum cryptographic protocols”. In: *J. Symb. Comput.* 73, pp. 192–220. URL: <https://api.semanticscholar.org/CorpusID:9696999>.
- Lean-FRO (Nov. 2020). *Lean manual*. URL: <https://lean-lang.org/lean4/doc/whatIsLean.html>.
- (Jan. 2021). *Lean manual*. URL: <https://lean-lang.org/lean4/doc/setup.html>.
- Leanprover-Community (May 2021). *Leanprover-community/Mathlib4: The Math Library of Lean 4*. URL: <https://github.com/leanprover-community/mathlib4>.
- Liu, Junyi, Bohua Zhan, Shuling Wang, Shenggang Ying, T. Liu, et al. (2019). “Quantum Hoare Logic”. In: *Arch. Formal Proofs* 2019. URL: <https://api.semanticscholar.org/CorpusID:85542178>.
- Liu, Junyi, Bohua Zhan, Shuling Wang, Shenggang Ying, Tao Liu, et al. (2019). “Formal Verification of Quantum Algorithms Using Quantum Hoare Logic”. In: *Computer Aided Verification*. Ed. by Isil Dillig and Serdar Tasiran. Cham: Springer International Publishing, pp. 187–207. ISBN: 978-3-030-25543-5.
- Mehta, Bhavik (Nov. 2023). *Formalising modern research mathematics in Real time*. URL: <https://xenaproject.wordpress.com/2023/11/04/formalising-modern-research-mathematics-in-real-time/>.
- Microsoft (Feb. 2023). *Lean*. URL: <https://www.microsoft.com/en-us/research/project/lean/>.
- NIST (Jan. 2017). *Post-quantum cryptography: CSRC*. URL: <https://csrc.nist.gov/projects/post-quantum-cryptography>.
- Paulson, Lawrence C. (1986). “Natural deduction as higher-order resolution”. In: *The Journal of Logic Programming* 3.3, pp. 237–258. ISSN: 0743-1066. DOI: [https://doi.org/10.1016/0743-1066\(86\)90015-4](https://doi.org/10.1016/0743-1066(86)90015-4). URL: <https://www.sciencedirect.com/science/article/pii/0743106686900154>.

- Peng, Yuxiang et al. (2022). “A formally certified end-to-end implementation of Shor’s factorization algorithm”. In: *Proceedings of the National Academy of Sciences of the United States of America* 120. URL: <https://api.semanticscholar.org/CorpusID:248177686>.
- Roy, Arnab, Anupam Datta, and John C. Mitchell (2007). “Formal Proofs of Cryptographic Security of Diffie-Hellman-Based Protocols”. In: *Trustworthy Global Computing*. URL: <https://api.semanticscholar.org/CorpusID:1502163>.
- Singhal, Kartik (2020). “Quantum Hoare Type Theory”. In: *QPL*. URL: <https://api.semanticscholar.org/CorpusID:215752888>.
- Standards, National Institute of and Technology (2023). *Module-Lattice-Based Digital Signature Standard*. Tech. rep. Federal Information Processing Standards Publications (FIPS PUBS) 204. Washington, D.C.: U.S. Department of Commerce. DOI: 10.6028/NIST.FIPS.204.
- The Coq Development Team (n.d.). *The Coq Proof Assistant*. URL: <https://coq.inria.fr/>.
- Zhou, Li et al. (Jan. 2023). “CoqQ: Foundational Verification of Quantum Programs”. In: *Proc. ACM Program. Lang.* 7.POPL. DOI: 10.1145/3571222. URL: <https://doi.org/10.1145/3571222>.
- Zhou, Yiqing, E. Miles Stoudenmire, and Xavier Waintal (Nov. 2020). “What Limits the Simulation of Quantum Computers?” In: *Phys. Rev. X* 10 (4), p. 041038. DOI: 10.1103/PhysRevX.10.041038. URL: <https://link.aps.org/doi/10.1103/PhysRevX.10.041038>.